Phase 2 Development Phase

Airbnb Database

DLBDSPBDM01 – Build a Data Mart in SQL

Tutor: Musharaf Doger

Name: Anita Skynar

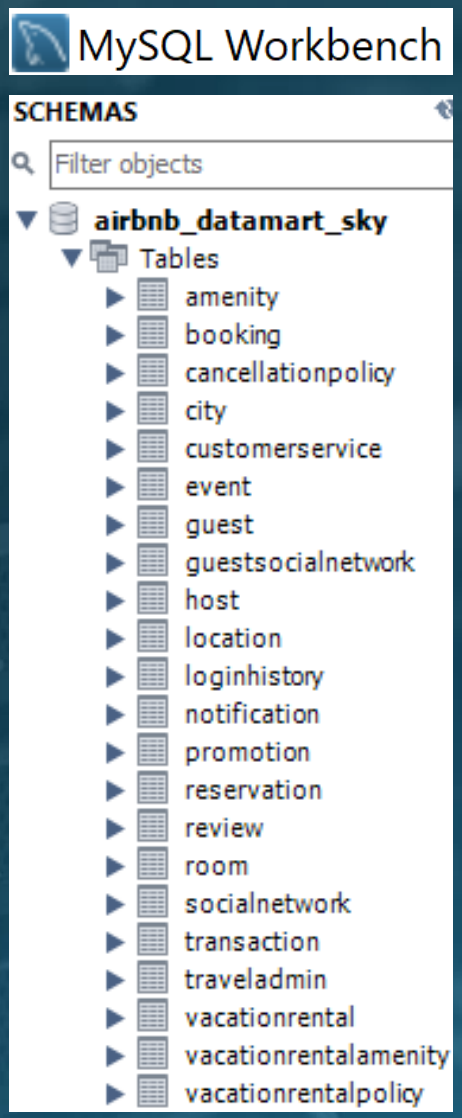
Matriculation: 92124577

Submission Date: 12 September 2024

Table of Contents

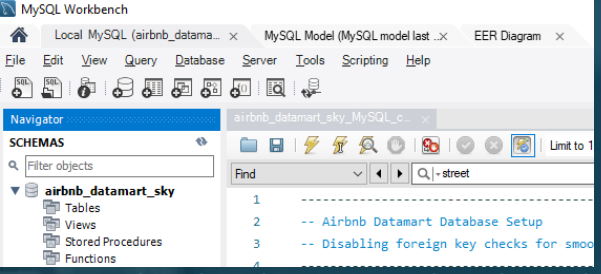
Introduction

Overview of the Airbnb Datamart Project:



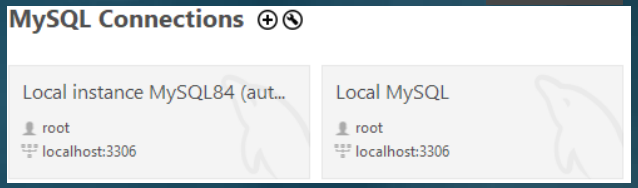
In the development phase of the Airbnb Datamart project, the database schema from the updated conception phase was implemented within the MySQL Workbench.

This included creating all tables and relationships as per the ERM Diagram and populating each table with at least 20 entries.



Setting Up MySQL Workbench

The database was developed using **MySQL Workbench**.



Steps to Run SQL Code in MySQL Workbench:

1.Open MySQL Workbench.

- Launch MySQL Workbench from your system.

2.Set up a New Connection:

- Click **+** under *MySQL Connections*.
- In the "Setup New Connection" window:
  - **Connection Name**: Choose any name (e.g., "Airbnb_DB").
  - **Hostname**: Use localhost (or your server IP if applicable).
  - **Port**: Default is 3306.
  - **Username**: Use root or another MySQL username.
  - **Password**: Click "Store in Vault" and enter the MySQL password.

3.Connect to Database:

- Click **Test Connection** to confirm it works.
- Then click **OK** to save and open the connection.

4.Run the SQL Code:

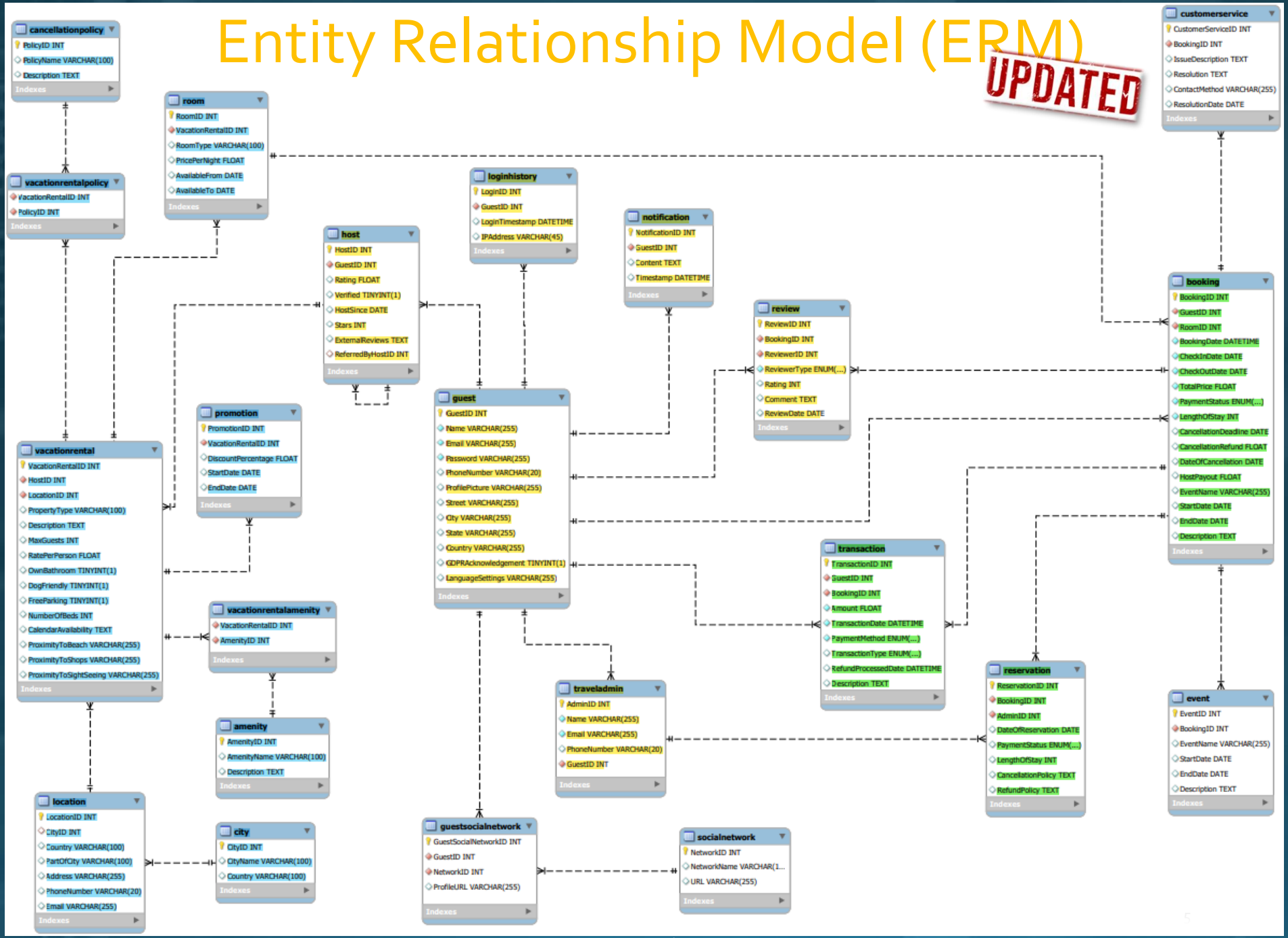
- Open your SQL file with all table creation, inserts, and test queries.
- Copy and paste the SQL code into the SQL editor or use **File > Open SQL Script** to load the file.
- Click the **Execute** button (lightning icon) to run the code.

5.Verify Tables and Data:

- Use SELECT * FROM queries to verify the data in each table.
- Ensure that all tables and relationships are created correctly.

Entity Relationship Model (ERM)

UPDATED



Database Setup

Initialization & Table Preparation:

To begin the implementation, it was necessary to prepare the database environment as shown in the code snippet.

Clean Environment:

Existing tables were dropped to avoid conflicts, ensuring a fresh start for the schema creation.

Foreign Key Checks:

Temporarily disabled during the drop process and re-enabled afterward to maintain integrity.

```
-- Disable foreign key checks to avoid issues during table creation
SET FOREIGN_KEY_CHECKS = 0;

-- Drop existing tables if they exist
DROP TABLE IF EXISTS GuestSocialNetwork;
DROP TABLE IF EXISTS SocialNetwork;
DROP TABLE IF EXISTS Notification;
DROP TABLE IF EXISTS Review;
DROP TABLE IF EXISTS CustomerService;
DROP TABLE IF EXISTS Reservation;
DROP TABLE IF EXISTS Transaction;
DROP TABLE IF EXISTS Booking;
DROP TABLE IF EXISTS Room;
DROP TABLE IF EXISTS VacationRentalPolicy;
DROP TABLE IF EXISTS CancellationPolicy;
DROP TABLE IF EXISTS VacationRentalAmenity;
DROP TABLE IF EXISTS Amenity;
DROP TABLE IF EXISTS VacationRental;
DROP TABLE IF EXISTS Location;
DROP TABLE IF EXISTS City;
DROP TABLE IF EXISTS LoginHistory;
DROP TABLE IF EXISTS TravelAdmin;
DROP TABLE IF EXISTS Host;
DROP TABLE IF EXISTS Guest;
DROP TABLE IF EXISTS Event;
DROP TABLE IF EXISTS Promotion;
```

Guest Table

GuestID	Name	Email	Password	PhoneNumber	ProfilePicture	Street	City	State	Cou
1	Hans Müller	hans.mueller@example.de	password111	491523456789	hans_profile.jpg	123 Main St	Berlin		Germ
2	Emma Karlsson	emma.karlsson@example.se	password112	467021234567	emma_profile.jpg	456 Elm St	Stockholm		Swe
3	Jean Dupont	jean.dupont@example.fr	password113	33122334455	jean_profile.jpg	789 Oak St	Paris		Fran
4	Giovanni Rossi	giovanni.rossi@example.it	password211	390123456789	giovanni_profile.jpg	101 Maple St	Rome		Italy

```sql
CREATE TABLE Guest (
    GuestID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(255) NOT NULL,
    Email VARCHAR(255) NOT NULL UNIQUE,
    Password VARCHAR(255) NOT NULL,
    PhoneNumber VARCHAR(20),
    ProfilePicture VARCHAR(255),
    Street VARCHAR(255), -- Added Street
    City VARCHAR(255),   -- Added City
    State VARCHAR(255),  -- Added State
    Country VARCHAR(255),-- Added Country
    GDPRAcknowledgement BOOLEAN,
    LanguageSettings VARCHAR(255)
);
```

1. Roles:

- **Guests**: Create and manage their own guest profiles.
- **Hosts**: Can view guest information for their bookings.
- **System Administrators**: Ensure data privacy and system-wide guest management.

2. Actions:

- **Guests**: Register, update, and manage their profiles.
- **Hosts**: View guest profiles related to their bookings.
- **System Administrators**: Maintain guest data privacy & manage guest information updates.

3. Data (summarized):

- **GuestID**: Unique identifier for each guest.
- **Personal Information**: Includes name, email (unique), phone number, and password.
- **Address Information**: Street, city, state, country.
- **Additional Info**: Profile picture, GDPR acknowledgement, language settings.

4. Relationships:

- **Guest - Booking**: A guest can have multiple bookings (one-to-many relationship with bookings).
- **Guest - GuestSocialNetwork**: A guest can be linked to multiple social network profiles through the GuestSocialNetwork table.

```sql
INSERT INTO Guest (Name, Email, Password, PhoneNumber, ProfilePicture, Street, City, State, Country, GDPRAcknowledgement, LanguageSettings)
VALUES
    ('Hans Müller', 'hans.mueller@example.de', 'password111', '491523456789', 'hans_profile.jpg', '123 Main St', 'Berlin', '', 'Germany', TRUE, 'German'),
    ('Emma Karlsson', 'emma.karlsson@example.se', 'password112', '467021234567', 'emma_profile.jpg', '456 Elm St', 'Stockholm', '', 'Sweden', TRUE, 'Swedish'),
    ('Jean Dupont', 'jean.dupont@example.fr', 'password113', '33122334455', 'jean_profile.jpg', '789 Oak St', 'Paris', '', 'France', TRUE, 'French'),
    ('Giovanni Rossi', 'giovanni.rossi@example.it', 'password211', '390123456789', 'giovanni_profile.jpg', '101 Maple St', 'Rome', '', 'Italy', TRUE, 'Italian'),
    ('Seo-jun Lee', 'seo.jun.lee@example.kr', 'password212', '821012345678', 'seo_profile.jpg', '202 Pine St', 'Seoul', '', 'South Korea', TRUE, 'Korean'),
    ('Sakura Tanaka', 'sakura.tanaka@example.jp', 'password213', '819012345678', 'sakura_profile.jpg', '303 Birch St', 'Tokyo', '', 'Japan', TRUE, 'Japanese'),
    ('Anna Svensson', 'anna.svensson@example.se', 'password100', '46709876543', 'anna_profile.jpg', '404 Cedar St', 'Gothenburg', '', 'Sweden', TRUE, 'Swedish'),
    ('Friedrich Becker', 'friedrich.becker@example.de', 'password101', '4915123456789', 'friedrich_profile.jpg', '505 Pine St', 'Munich', '', 'Germany', TRUE, 'German'),
    ('Pierre Martin', 'pierre.martin@example.fr', 'password110', '33122334455', 'pierre_profile.jpg', '606 Maple St', 'Lyon', '', 'France', TRUE, 'French'),
    ('Alessandro Bianchi', 'alessandro.bianchi@example.it', 'password200', '390223344556', 'alessandro_profile.jpg', '707 Oak St', 'Milan', '', 'Italy', TRUE, 'Italian'),
    ('Ahmed El-Sayed', 'ahmed.el-sayed@example.eg', 'password202', '201234567890', 'ahmed_profile.jpg', '808 Pine St', 'Cairo', '', 'Egypt', TRUE, 'Arabic'),
    ('John Smith', 'john.smith@example.co.uk', 'password201', '441234567890', 'john_profile.jpg', '909 Birch St', 'London', '', 'United Kingdom', TRUE, 'English (UK)'),
    ('Olivia Brown', 'olivia.brown@example.au', 'password220', '611234567890', 'olivia_profile.jpg', '1010 Cedar St', 'Sydney', '', 'Australia', TRUE, 'English (Australia)'),
    ('Michael Johnson', 'michael.johnson@example.us', 'password203', '11234567890', 'michael_profile.jpg', '1111 Maple St', 'New York', 'NY', 'USA', TRUE, 'English (USA)'),
    ('Emma Williams', 'emma.williams@example.ca', 'password103', '11234567891', 'emma_profile.jpg', '1212 Oak St', 'Toronto', '', 'Canada', TRUE, 'English (Canada)'),
    ('Chloe Dubois', 'chloe.dubois@example.ch', 'password301', '33122334456', 'chloe_profile.jpg', '1313 Pine St', 'Geneva', '', 'Switzerland', TRUE, 'French (Switzerland)'),
    ('Mats Andersson', 'mats.andersson@example.se', 'password300', '46709876544', 'mats_profile.jpg', '1414 Cedar St', 'Malmö', '', 'Sweden', TRUE, 'Swedish'),
    ('Sophie Schmidt', 'sophie.schmidt@example.de', 'password302', '491523456789', 'sophie_profile.jpg', '1515 Maple St', 'Hamburg', '', 'Germany', TRUE, 'German'),
    ('Luca Bruni', 'luca.bruni@example.it', 'password311', '39034567890', 'luca_profile.jpg', '1616 Oak St', 'Florence', '', 'Italy', TRUE, 'Italian'),
    ('Isabelle Laurent', 'isabelle.laurent@example.fr', 'password321', '33122334455', 'isabelle_profile.jpg', '1717 Pine St', 'Nice', '', 'France', TRUE, 'French');
```

Host Table

```sql
CREATE TABLE Host (
    HostID INT AUTO_INCREMENT PRIMARY KEY,
    GuestID INT NOT NULL,
    Rating FLOAT,
    Verified BOOLEAN,
    HostSince DATE,
    Stars INT,
    ExternalReviews TEXT,
    ReferredByHostID INT NULL,
    FOREIGN KEY (GuestID) REFERENCES Guest(GuestID) ON DELETE CASCADE, -- Add cascade delete
    FOREIGN KEY (ReferredByHostID) REFERENCES Host(HostID) ON DELETE SET NULL -- -- Recursive relationship, Avoid circular referenc
);
```

1. Roles:

- **Hosts**: Manage their profiles, guest interactions and ratings.
- **Guests**: View host details when booking & reviews.
- **System Administrators**: Verify hosts and manage reviews and ensuring referrals between hosts.

2. Actions:

- **Hosts**: Update profiles, ratings, and refer other hosts.
- **Guests**: View host profiles and ratings.
- **System Administrators**: Verify hosts and maintain referral relationships.

3. Data (summarized):

- **HostID**: PK, uniquely identifying each host.
- **GuestID**: Links hosts to a guest profile (host = guest).
- **Rating/Stars**: Host's review score & ratings from guests
- **Verified**: Host´s verification status.
- **HostSince**: Date when hosting started.
- **ReferredByHostID**: Tracks host referrals.

4. Relationships:

- **Host - Guest**: Each host is also a guest, creating a one-to-one relationship between the two tables.
- **Host - Host**: Hosts can refer other hosts, establishing a self-referencing (recursive) relationship. This relationship is managed carefully to avoid circular references by using SET NULL if the referring host is deleted.

```sql
INSERT INTO Host (GuestID, Rating, Verified, HostSince, Stars, ExternalReviews, ReferredByH
VALUES
    (1, 4.5, TRUE, '2020-01-01', 5, 'Great reviews from external sites.', NULL),
    (2, 4.0, TRUE, '2021-03-15', 4, 'Positive feedback from external platforms.', NULL),
    (3, 4.8, TRUE, '2019-11-20', 5, 'Outstanding reviews.', NULL),
    (4, 4.2, TRUE, '2020-06-25', 4, 'Excellent ratings on various platforms.', NULL),
    (5, 4.7, TRUE, '2021-07-10', 5, 'Highly recommended by guests.', NULL),
    (6, 4.9, TRUE, '2021-05-22', 5, 'Top-rated host with many positive reviews.', NULL),
    (7, 4.3, TRUE, '2019-12-05', 4, 'Good reviews overall.', NULL),
    (8, 4.6, TRUE, '2020-02-15', 5, 'Consistent positive feedback from guests.', NULL),
    (9, 4.1, TRUE, '2021-09-01', 4, 'Mixed reviews but generally positive.', NULL),
    (10, 4.4, TRUE, '2021-08-18', 5, 'Great experience reported by guests.', NULL),
    (11, 4.3, TRUE, '2020-10-10', 4, 'Strong feedback from external reviews.', NULL),
    (12, 4.8, TRUE, '2021-01-01', 5, 'Highly recommended by several sources.', NULL),
    (13, 4.7, TRUE, '2020-03-20', 5, 'Great host with excellent reviews.', NULL),
    (14, 4.9, TRUE, '2019-07-07', 5, 'One of the top-rated hosts.', NULL),
    (15, 4.5, TRUE, '2021-04-15', 5, 'Guests consistently leave positive feedback.', NULL),
    (16, 4.6, TRUE, '2019-09-25', 5, 'Highly rated by multiple sources.', NULL),
    (17, 4.2, TRUE, '2020-11-10', 4, 'Reviews indicate a good experience.', NULL),
    (18, 4.5, TRUE, '2021-02-22', 5, 'Popular host with lots of positive reviews.', NULL),
    (19, 4.7, TRUE, '2020-12-15', 5, 'Guests frequently recommend this host.', NULL),
    (20, 4.4, TRUE, '2021-03-10', 4, 'Well-reviewed host with solid ratings.', NULL);
```

HostID	GuestID	Rating	Verified	HostSince	Stars	ExternalReviews	Referred
1	1	4.5	1	2020-01-01	5	Great reviews from external sites.	NULL
2	2	4	1	2021-03-15	4	Positive feedback from external platforms.	NULL
3	3	4.8	1	2019-11-20	5	Outstanding reviews.	NULL
4	4	4.2	1	2020-06-25	4	Excellent ratings on various platforms.	NULL

Booking Table

1. Roles:

- **Guests**: Make and manage bookings.
- **Hosts**: Monitor bookings and handle cancellations.
- **System Administrators**: Oversee refunds and cancellations.

2. Actions:

- **Guests**: Create, update, cancel bookings.
- **Hosts**: Manage bookings and refunds.
- **System Administrators**: Process refunds and maintain records.

3. Data (summarized):

- **Guest & Room Info**: Links the booking to a guest (GuestID)/room (RoomID).
- **Booking & Stay Details:** booking date, check-in/check-out, stay length.
- **Payment Status**: Shows total price, paid, pending, or canceled status.
- **Cancellations/Refunds**: Handles deadlines and refunds.
- **Host & Event Info**: Details such as host payout, event name, and event dates.

4. Relationships:

- **Guest - Booking**: One guest, multiple bookings.
- **Room - Booking**: One room, many bookings.
- **Transaction - Booking**: Tracks payments and refunds.

```sql
CREATE TABLE Booking (
    BookingID INT AUTO_INCREMENT PRIMARY KEY,
    GuestID INT NOT NULL,
    RoomID INT NOT NULL,
    BookingDate DATETIME NOT NULL,
    CheckInDate DATE NOT NULL,
    CheckOutDate DATE NOT NULL,
    TotalPrice FLOAT NOT NULL,
    PaymentStatus ENUM('Paid', 'Pending', 'Cancelled') NOT NULL,
    LengthOfStay INT NOT NULL,
    CancellationDeadline DATE,
    CancellationRefund FLOAT,
    DateOfCancellation DATE,
    HostPayout FLOAT,
    EventName VARCHAR(255),
    StartDate DATE,
    EndDate DATE,
    Description TEXT,
    FOREIGN KEY (GuestID) REFERENCES Guest(GuestID),
    FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);
```

```sql
INSERT INTO Booking (GuestID, RoomID, BookingDate, CheckInDate, CheckOutDate, TotalPrice, PaymentStatus, LengthOfStay, CancellationDeadline, CancellationRefund, DateOfCancellation, HostPayout, EventName, StartDate, EndDate, De
VALUES
    (1, 1, NOW(), '2024-09-01', '2024-09-07', 900.00, 'Pending', 6, '2024-08-29', 800.00, NULL, 850.00, 'Berlin Marathon', '2024-09-01', '2024-09-07', 'Running event in Berlin'),
    (2, 2, NOW(), '2024-09-05', '2024-09-10', 600.00, 'Paid', 5, '2024-09-03', 550.00, NULL, 580.00, 'Stockholm Music Festival', '2024-09-05', '2024-09-10', 'Music festival in Stockholm'),
    (3, 3, NOW(), '2024-09-10', '2024-09-15', 750.00, 'Pending', 5, '2024-09-08', 700.00, NULL, 725.00, 'Paris Fashion Week', '2024-09-10', '2024-09-15', 'Fashion event in Paris'),
    (4, 4, NOW(), '2024-09-12', '2024-09-18', 1500.00, 'Pending', 6, '2024-09-10', 1400.00, NULL, 1450.00, 'Rome Film Festival', '2024-09-12', '2024-09-18', 'Film festival in Rome'),
    (5, 5, NOW(), '2024-09-15', '2024-09-20', 450.00, 'Paid', 5, '2024-09-13', 400.00, NULL, 425.00, 'Seoul Food Festival', '2024-09-15', '2024-09-20', 'Food festival in Seoul'),
    (6, 6, NOW(), '2024-09-18', '2024-09-22', 560.00, 'Cancelled', 4, '2024-09-16', 500.00, '2024-09-18', 530.00, 'Tokyo Game Show', '2024-09-18', '2024-09-22', 'Gaming event in Tokyo'),
    (7, 7, NOW(), '2024-09-20', '2024-09-25', 650.00, 'Paid', 5, '2024-09-18', 600.00, NULL, 620.00, 'Gothenburg Book Fair', '2024-09-20', '2024-09-25', 'Book fair in Gothenburg'),
    (8, 8, NOW(), '2024-09-25', '2024-09-30', 700.00, 'Pending', 5, '2024-09-23', 650.00, NULL, 680.00, 'Munich Oktoberfest', '2024-09-25', '2024-09-30', 'Festival in Munich'),
    (9, 9, NOW(), '2024-09-28', '2024-10-03', 800.00, 'Paid', 5, '2024-09-26', 750.00, NULL, 780.00, 'Lyon Lights Festival', '2024-09-28', '2024-10-03', 'Light festival in Lyon'),
    (10, 10, NOW(), '2024-10-01', '2024-10-05', 480.00, 'Pending', 4, '2024-09-29', 400.00, NULL, 450.00, 'Milan Fashion Week', '2024-10-01', '2024-10-05', 'Fashion event in Milan'),
    (11, 11, NOW(), '2024-10-03', '2024-10-08', 425.00, 'Paid', 5, '2024-10-01', 380.00, NULL, 410.00, 'Cairo Film Festival', '2024-10-03', '2024-10-08', 'Film festival in Cairo'),
    (12, 12, NOW(), '2024-10-07', '2024-10-12', 1000.00, 'Pending', 5, '2024-10-05', 950.00, NULL, 975.00, 'London Literature Festival', '2024-10-07', '2024-10-12', 'Literature festival in London'),
    (13, 13, NOW(), '2024-10-10', '2024-10-15', 1500.00, 'Paid', 5, '2024-10-08', 1400.00, NULL, 1450.00, 'Sydney Opera Festival', '2024-10-10', '2024-10-15', 'Opera festival in Sydney'),
    (14, 14, NOW(), '2024-10-12', '2024-10-17', 800.00, 'Pending', 5, '2024-10-10', 750.00, NULL, 780.00, 'New York Film Festival', '2024-10-12', '2024-10-17', 'Film festival in New York'),
    (15, 15, NOW(), '2024-10-15', '2024-10-20', 500.00, 'Paid', 5, '2024-10-13', 450.00, NULL, 475.00, 'Toronto Beer Festival', '2024-10-15', '2024-10-20', 'Beer festival in Toronto'),
    (16, 16, NOW(), '2024-10-18', '2024-10-22', 1200.00, 'Cancelled', 4, '2024-10-16', 1100.00, '2024-10-18', 1150.00, 'Zurich International Film Festival', '2024-10-18', '2024-10-22', 'Film festival in Zurich'),
    (17, 17, NOW(), '2024-10-20', '2024-10-25', 650.00, 'Pending', 5, '2024-10-18', 600.00, NULL, 620.00, 'Malmö Food Fair', '2024-10-20', '2024-10-25', 'Food festival in Malmö'),
    (18, 18, NOW(), '2024-10-25', '2024-10-30', 350.00, 'Paid', 5, '2024-10-23', 300.00, NULL, 330.00, 'Hamburg Jazz Festival', '2024-10-25', '2024-10-30', 'Jazz festival in Hamburg'),
    (19, 19, NOW(), '2024-10-28', '2024-11-02', 1680.00, 'Pending', 5, '2024-10-26', 1600.00, NULL, 1650.00, 'Florence Art Exhibition', '2024-10-28', '2024-11-02', 'Art exhibition in Florence'),
    (20, 20, NOW(), '2024-11-01', '2024-11-05', 525.00, 'Paid', 4, '2024-10-30', 500.00, NULL, 510.00, 'Nice Film Festival', '2024-11-01', '2024-11-05', 'Film festival in Nice');
```

BookingID	GuestID	RoomID	BookingDate	CheckInDate	CheckOutDate	TotalPrice	PaymentStatus	LengthOfStay	CancellationDeadline	CancellationRefund	DateOfCancellation	HostPayout	EventName	StartDate	EndDate	Description
1	1	1	2024-09-09 22:30:21	2024-09-01	2024-09-07	900	Pending	6	2024-08-29	800	NULL	850	Berlin Marathon	2024-09-01	2024-09-07	Running event in Berli
2	2	2	2024-09-09 22:30:21	2024-09-05	2024-09-10	600	Paid	5	2024-09-03	550	NULL	580	Stockholm Music Festival	2024-09-05	2024-09-10	Music festival in Stockl
3	3	3	2024-09-09 22:30:21	2024-09-10	2024-09-15	750	Pending	5	2024-09-08	700	NULL	725	Paris Fashion Week	2024-09-10	2024-09-15	Fashion event in Paris
4	4	4	2024-09-09 22:30:21	2024-09-12	2024-09-18	1500	Pending	6	2024-09-10	1400	NULL	1450	Rome Film Festival	2024-09-12	2024-09-18	Film festival in Rome
5	5	5	2024-09-09 22:30:21	2024-09-15	2024-09-20	450	Paid	5	2024-09-13	400	NULL	425	Seoul Food Festival	2024-09-15	2024-09-20	Food festival in Seoul

TravelAdmin Table

```sql
CREATE TABLE TravelAdmin (
    AdminID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(255) NOT NULL,
    Email VARCHAR(255) NOT NULL UNIQUE,
    PhoneNumber VARCHAR(20),
    GuestID INT NOT NULL,
    FOREIGN KEY (GuestID) REFERENCES Guest(GuestID)
);
```

```sql
INSERT INTO TravelAdmin (Name, Email, PhoneNumber, GuestID)
VALUES
    ('Hans Müller', 'hans.mueller@example.de', '+49 491523456789', 1),
    ('Emma Karlsson', 'emma.karlsson@example.se', '+46 467021234567', 2),
    ('Jean Dupont', 'jean.dupont@example.fr', '+33 33122334455', 3),
    ('Giovanni Rossi', 'giovanni.rossi@example.it', '+39 390123456789', 4),
    ('Seo-jun Lee', 'seo.jun.lee@example.kr', '+82 821012345678', 5),
    ('Sakura Tanaka', 'sakura.tanaka@example.jp', '+81 819012345678', 6),
    ('Anna Svensson', 'anna.svensson@example.se', '+46 46709876543', 7),
    ('Friedrich Becker', 'friedrich.becker@example.de', '+49 491523456789', 8),
    ('Pierre Martin', 'pierre.martin@example.fr', '+33 33122334455', 9),
    ('Alessandro Bianchi', 'alessandro.bianchi@example.it', '+39 390223344556', 10),
    ('Ahmed El-Sayed', 'ahmed.el-sayed@example.eg', '+20 201234567890', 11),
    ('John Smith', 'john.smith@example.co.uk', '+44 441234567890', 12),
    ('Olivia Brown', 'olivia.brown@example.au', '+61 611234567890', 13),
    ('Michael Johnson', 'michael.johnson@example.ca', '+1 11234567890', 14),
    ('Emma Williams', 'emma.williams@example.ca', '+1 11234567890', 15),
    ('Chloe Dubois', 'chloe.dubois@example.ch', '+41 46709876543', 16),
    ('Mats Andersson', 'mats.andersson@example.se', '+46 46709876544', 17),
    ('Sophie Schmidt', 'sophie.schmidt@example.de', '+49 491523456789', 18),
    ('Luca Bruni', 'luca.bruni@example.it', '+39 390234567890', 19),
    ('Isabelle Laurent', 'isabelle.laurent@example.fr', '+33 33122334455', 20);
```

AdminID	Name	Email	PhoneNumber	GuestID
1	Hans Müller	hans.mueller@example.de	+49 491523456789	1
2	Emma Karlsson	emma.karlsson@example.se	+46 467021234567	2
3	Jean Dupont	jean.dupont@example.fr	+33 33122334455	3
4	Giovanni Rossi	giovanni.rossi@example.it	+39 390123456789	4

1. Roles:

- **Travel Administrators:** Oversee guest bookings, cancellations, and other administrative tasks.
- **System Administrators:** Oversee travel admin access and maintain their records.

2. Actions:

- **Travel Administrators:** Manage bookings and policies on behalf of guests.
- **System Administrators:** Set up travel admin accounts and manage access.

3. Data (summarized):

- **AdminID:** Unique identifier for the travel admin.
- **GuestID:** Links each travel admin to a specific guest account for identity purposes.
- **Contact Information:** Includes name, email, and phone number.

4. Relationships:

- **Guest - TravelAdmin:** Each travel admin is associated with a guest account for identity verification.

Reservation Table

```sql
CREATE TABLE Reservation (
    ReservationID INT AUTO_INCREMENT PRIMARY KEY,
    BookingID INT NOT NULL,
    AdminID INT NOT NULL,
    DateOfReservation DATE,
    PaymentStatus ENUM('Paid', 'Pending', 'Cancelled'),
    LengthOfStay INT,
    CancellationPolicy TEXT,
    RefundPolicy TEXT,
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID),
    FOREIGN KEY (AdminID) REFERENCES TravelAdmin(AdminID)
);
```

```sql
INSERT INTO Reservation (BookingID, AdminID, DateOfReservation, PaymentStatus, LengthOfStay, CancellationPolicy, RefundPolicy)
VALUES
    (1, 1, '2024-09-01', 'Pending', 7, 'Flexible', 'Full refund up to 1 day before arrival'),
    (2, 2, '2024-09-05', 'Paid', 5, 'Moderate', 'Full refund up to 5 days before arrival'),
    (3, 3, '2024-09-10', 'Cancelled', 5, 'Strict', '50% refund up to 7 days before arrival'),
    (4, 4, '2024-09-12', 'Pending', 6, 'Super Strict', '50% refund up to 14 days before arrival'),
    (5, 5, '2024-09-15', 'Paid', 5, 'Non-refundable', 'No refund after booking'),
    (6, 6, '2024-09-18', 'Cancelled', 4, 'Relaxed', 'Full refund up to 3 days before arrival'),
    (7, 7, '2024-09-20', 'Paid', 5, 'Firm', '75% refund up to 2 days before arrival'),
    (8, 8, '2024-09-25', 'Pending', 5, 'Standard', '50% refund up to 4 days before arrival'),
    (9, 9, '2024-09-28', 'Paid', 5, 'Special', 'Full refund up to 10 days before arrival'),
    (10, 10, '2024-10-01', 'Pending', 4, 'Custom', 'Refund terms are set by the host'),
    (11, 11, '2024-10-03', 'Paid', 5, 'Partial Refund', '30% refund up to 3 days before arrival'),
    (12, 12, '2024-10-07', 'Pending', 5, 'Full Refund', '100% refund up to 7 days before arrival'),
    (13, 13, '2024-10-10', 'Paid', 5, 'Weekend Special', 'No refund on weekends'),
    (14, 14, '2024-10-12', 'Pending', 5, 'Holiday Refund', 'Full refund on holidays'),
    (15, 15, '2024-10-15', 'Paid', 5, 'Seasonal Refund', 'Partial refund depending on the season'),
    (16, 16, '2024-10-18', 'Cancelled', 4, 'Last Minute', 'No refund for last-minute bookings'),
    (17, 17, '2024-10-20', 'Pending', 5, 'High Season', 'Strict refund policy during high season'),
    (18, 18, '2024-10-25', 'Paid', 5, 'Event Special', 'No refund during special events'),
    (19, 19, '2024-10-28', 'Pending', 5, 'Flexible Plus', 'Full refund up to 2 days before arrival'),
    (20, 20, '2024-11-01', 'Paid', 4, 'Summer Special', 'Full refund up to 7 days before arrival');
```

ReservationID	BookingID	AdminID	DateOfReservation	PaymentStatus	LengthOfStay	CancellationPolicy	RefundPolicy
1	1	1	2024-09-01	Pending	7	Flexible	Full refund up
2	2	2	2024-09-05	Paid	5	Moderate	Full refund up
3	3	3	2024-09-10	Cancelled	5	Strict	50% refund u
4	4	4	2024-09-12	Pending	6	Super Strict	50% refund u
5	5	5	2024-09-15	Paid	5	Non-refundable	No refund aft

1. Roles:

- **Guests**: Make reservations and manage payments.
- **Travel Admins**: Monitor and manage reservations and cancellations.
- **System Administrators**: Maintain reservation system integrity and handle platform issues.

2. Actions:

- **Guests**: Create, update, or cancel reservations.
- **Travel Admins**: Process and manage reservations, handle refunds, and apply cancellation policies.
- **System Administrators**: Maintain system for booking, track cancellations and refunds.

3. Data (summarized):

- **ReservationID**: Unique ID for the reservation.
- **BookingID**: Links to a specific booking.
- **AdminID**: References the Travel Admin managing the reservation.
- **Reservation Details**: Date of reservation, payment status, length of stay, cancellation and refund policies.

4. Relationships:

- **Booking - Reservation**: Each reservation is linked to a booking.
- **Admin - Reservation**: Admins are linked to reservations they manage.

Transaction Table

By managing both payments and refunds in one table, it simplifies the payment process and reduces redundancy, ensuring accurate financial tracking for each booking.

```sql
CREATE TABLE Transaction (
    TransactionID INT AUTO_INCREMENT PRIMARY KEY,
    GuestID INT NOT NULL,
    BookingID INT NOT NULL,
    Amount FLOAT NOT NULL,
    TransactionDate DATETIME NOT NULL,
    PaymentMethod ENUM('CreditCard', 'BankTransfer', 'Cash', 'Voucher', 'PayPal', 'ApplePay', 'GPay', 'CreditCard_MasterCa
    'CreditCard_Visa', 'CreditCard_AMEX', 'CreditCard_FirstCard', 'CreditCard_DinersClub', 'Maestro', 'SOFORT_Payment', 'B
    TransactionType ENUM('Payment', 'Refund') NOT NULL,  -- Column to specify if the transaction is a payment or refund
    RefundProcessedDate DATETIME NULL,  -- Column to specify when the refund was processed (can be NULL for payments)
    Description TEXT NOT NULL,  -- Description field to summarize the transaction (e.g., refund reason)
    FOREIGN KEY (GuestID) REFERENCES Guest(GuestID),
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)
);
```

```sql
INSERT INTO Transaction (GuestID, BookingID, Amount, TransactionDate, PaymentMethod, TransactionType, RefundProcessedDate, Desc
VALUES
    (1, 1, 900.00, '2024-09-01', 'CreditCard_Visa', 'Payment', NULL, 'Booking for vacation rental'),
    (2, 2, 600.00, '2024-09-05', 'PayPal', 'Payment', NULL, 'Booking for vacation rental'),
    (3, 3, 750.00, '2024-09-10', 'CreditCard_MasterCard', 'Payment', NULL, 'Booking for vacation rental'),
    (4, 4, 1500.00, '2024-09-12', 'BankTransfer', 'Payment', NULL, 'Booking for vacation rental'),
    (5, 5, 450.00, '2024-09-15', 'CreditCard_AMEX', 'Payment', NULL, 'Booking for vacation rental'),
    (6, 6, 560.00, '2024-09-18', 'ApplePay', 'Payment', NULL, 'Booking for vacation rental'),
    (7, 7, 650.00, '2024-09-20', 'CreditCard_Visa', 'Payment', NULL, 'Booking for vacation rental'),
    (8, 8, 700.00, '2024-09-25', 'GPay', 'Payment', NULL, 'Booking for vacation rental'),
    (9, 9, 800.00, '2024-09-28', 'Maestro', 'Payment', NULL, 'Booking for vacation rental'),
    (10, 10, 480.00, '2024-10-01', 'PayPal', 'Payment', NULL, 'Booking for vacation rental'),
    (11, 11, 200.00, '2024-09-02', 'CreditCard_Visa', 'Refund', '2024-09-02 10:00:00', 'Partial refund for early cancellation'),
    (12, 12, 150.00, '2024-09-06', 'PayPal', 'Refund', '2024-09-06 14:00:00', 'Refund due to service issues'),
    (13, 13, 100.00, '2024-09-11', 'CreditCard_MasterCard', 'Refund', '2024-09-11 16:30:00', 'Refund for service issues'),
    (14, 14, 1500.00, '2024-09-13', 'BankTransfer', 'Refund', '2024-09-13 12:00:00', 'Full refund for booking cancellation'),
    (15, 15, 450.00, '2024-09-16', 'CreditCard_AMEX', 'Refund', '2024-09-16 10:00:00', 'Full refund for booking cancellation'),
    (16, 16, 560.00, '2024-09-18', 'ApplePay', 'Refund', '2024-09-18 12:00:00', 'Refund for booking cancellation'),
    (17, 17, 650.00, '2024-09-20', 'CreditCard_Visa', 'Refund', '2024-09-20 09:00:00', 'Refund due to service issues'),
    (18, 18, 700.00, '2024-09-25', 'GPay', 'Refund', '2024-09-25 15:00:00', 'Partial refund for service issues'),
    (19, 19, 800.00, '2024-09-28', 'Maestro', 'Refund', '2024-09-28 17:00:00', 'Refund for booking cancellation'),
    (20, 20, 525.00, '2024-10-01', 'PayPal', 'Refund', '2024-10-01 11:30:00', 'Full refund for booking cancellation');
```

TransactionID	GuestID	BookingID	Amount	TransactionDate	PaymentMethod	TransactionType	RefundProcessedDate	Description
1	1	1	900	2024-09-01 00:00:00	CreditCard_Visa	Payment	NULL	Booking for
2	2	2	600	2024-09-05 00:00:00	PayPal	Payment	NULL	Booking for
3	3	3	750	2024-09-10 00:00:00	CreditCard_MasterCard	Payment	NULL	Booking for
4	4	4	1500	2024-09-12 00:00:00	BankTransfer	Payment	NULL	Booking for

1. Roles:

- **Guests:** Can make payments and receive refunds related to their bookings.
- **Hosts:** Not directly involved but can view payment statuses of guests.
- **System Administrators:** Manage payment processing and ensure refunds are correctly issued.

2. Actions:

- **Guests:** Initiate payments or request refunds.
- **System Administrators:** Process payments, handle refunds, and ensure transaction records are up to date.

3. Data (summarized):

- **TransactionID:** Unique identifier for each transaction.
- **GuestID & BookingID:** Links the transaction to a specific guest and booking.
- **Amount:** The total value of the transaction.
- **TransactionDate & RefundDate:** Tracks when transactions occur, including refund processing dates.
- **TransactionMethod & Type:** Specifies payment method and whether it's a payment or refund.

4. Relationships:

- **Guest - Transaction:** Each guest can have multiple transactions.
- **Booking - Transaction:** Each booking can be associated with multiple transactions.

Review Table

```sql
CREATE TABLE Review (
    ReviewID INT AUTO_INCREMENT PRIMARY KEY,
    BookingID INT NOT NULL,
    ReviewerID INT NOT NULL,  -- ID of either guest or host
    ReviewerType ENUM('Guest', 'Host') NOT NULL, -- Distinguish whether review is from a guest or hos
    Rating INT,
    Comment TEXT,
    ReviewDate DATE,
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID) ON DELETE CASCADE, -- Add cascade delete
    FOREIGN KEY (ReviewerID) REFERENCES Guest(GuestID) ON DELETE CASCADE  -- Assume Host is a Guest
);
```

```sql
INSERT INTO Review (BookingID, ReviewerID, ReviewerType, Rating, Comment, ReviewDate)
VALUES
    (1, 1, 'Guest', 5, 'Fantastisches Erlebnis, werde definitiv zurückkommen!', '2024-09-01'),
    (2, 2, 'Host', 4, 'Bra gäst, men lite bullrigt ibland.', '2024-09-02'), -- Swedish
    (3, 3, 'Guest', 5, 'Expérience merveilleuse, très propre!', '2024-09-03'), -- French
    (4, 4, 'Host', 3, 'Buon soggiorno, ma il Wi-Fi era un po\' lento.', '2024-09-04'), -- Ita
    (5, 5, 'Guest', 4, '훌륭한 숙박이었어요!', '2024-09-05'), -- Korean
    (6, 6, 'Host', 5, '最高の滞在！また利用したいです。', '2024-09-06'), -- Japanese
    (7, 7, 'Guest', 2, 'La habitación era más pequeña de lo esperado.', '2024-09-07'), -- Spa
    (8, 8, 'Host', 4, 'Локация отличная, хорошие гости.', '2024-09-08'), -- Russian
    (9, 9, 'Guest', 5, 'Wonderful stay, highly recommended!', '2024-09-09'), -- English
    (10, 10, 'Host', 3, 'Valor razonable, pero con algunos problemas de mantenimiento.', '202
    (11, 11, 'Guest', 5, 'Ottima esperienza!', '2024-09-11'), -- Italian
    (12, 12, 'Host', 4, 'Le client était respectueux et courtois.', '2024-09-12'), -- French
    (13, 13, 'Guest', 5, 'Great stay, the place was spotless!', '2024-09-13'), -- English
    (14, 14, 'Host', 3, 'L\'invitato è arrivato in ritardo.', '2024-09-14'), -- Italian
    (15, 15, 'Guest', 4, 'Gästvänligt ställe, men något bullrigt.', '2024-09-15'), -- Swedish
    (16, 16, 'Host', 5, 'Excelente huésped, muy comunicativo.', '2024-09-16'), -- Spanish
    (17, 17, 'Guest', 2, 'El apartamento no cumplió con mis expectativas.', '2024-09-17'), --
    (18, 18, 'Host', 4, 'Great guest, very tidy and respectful.', '2024-09-18'), -- English
    (19, 19, 'Guest', 5, 'حجز رائع، سأعود مرة أخرى!', '2024-09-19'), -- Arabic
    (20, 20, 'Host', 3, 'L\'invité aurait pu être plus réactif.', '2024-09-20'); -- French
```

ReviewID	BookingID	ReviewerID	ReviewerType	Rating	Comment	ReviewDate
1	1	1	Guest	5	Fantastisches Erlebnis, werde definitiv z...	2024-09-01
2	2	2	Host	4	Bra gäst, men lite bullrigt ibland.	2024-09-02
3	3	3	Guest	5	Expérience merveilleuse, très propre!	2024-09-03
4	4	4	Host	3	Buon soggiorno, ma il Wi-Fi era un po' le...	2024-09-04
5	5	5	Guest	4	훌륭한 숙박이었어요!	2024-09-05

1. Roles:

- **Guests**: Provide reviews for stays.
- **Hosts**: Leave feedback on guests.
- **System Administrators**: Manage review system and ensure quality control.

2. Actions:

- **Guests**: Write reviews for their bookings.
- **Hosts**: Review their guests' behavior and experience.
- **System Administrators**: Ensure review integrity and manage feedbacks.

3. Data (summarized):

- **ReviewID**: Unique ID for each review.
- **BookingID**: Links the review to a specific booking.
- **ReviewerID**: The ID of the reviewer (either guest or host).
- **Review Details**: Type (guest or host), rating, and comment with date.

4. Relationships:

- **Booking - Review**: A booking can have multiple reviews from guests and hosts.
- **Reviewer - Review**: Reviews are linked to the user who left the review, whether guest or host.

CancellationPolicy Table

```sql
CREATE TABLE CancellationPolicy (
    PolicyID INT AUTO_INCREMENT PRIMARY KEY,
    PolicyName VARCHAR(100),
    Description TEXT
);
```

```sql
INSERT INTO CancellationPolicy (PolicyName, Description)
VALUES
    ('Flexible', 'Full refund up to 1 day before arrival'),
    ('Moderate', 'Full refund up to 5 days before arrival'),
    ('Strict', '50% refund up to 7 days before arrival'),
    ('Super Strict', '50% refund up to 14 days before arrival'),
    ('Non-refundable', 'No refund after booking'),
    ('Relaxed', 'Full refund up to 3 days before arrival'),
    ('Firm', '75% refund up to 2 days before arrival'),
    ('Standard', '50% refund up to 4 days before arrival'),
    ('Special', 'Full refund up to 10 days before arrival'),
    ('Custom', 'Refund terms are set by the host'),
    ('Partial Refund', '30% refund up to 3 days before arrival'),
    ('Full Refund', '100% refund up to 7 days before arrival'),
    ('Weekend Special', 'No refund on weekends'),
    ('Holiday Refund', 'Full refund on holidays'),
    ('Seasonal Refund', 'Partial refund depending on the season'),
    ('Last Minute', 'No refund for last-minute bookings'),
    ('High Season', 'Strict refund policy during high season'),
    ('Event Special', 'No refund during special events'),
    ('Flexible Plus', 'Full refund up to 2 days before arrival'),
    ('Summer Special', 'Full refund up to 7 days before arrival');
```

PolicyID	PolicyName	Description
1	Flexible	Full refund up to 1 day before arrival
2	Moderate	Full refund up to 5 days before arrival
3	Strict	50% refund up to 7 days before arrival
4	Super Strict	50% refund up to 14 days before arrival

The **CancellationPolicy** table defines different cancellation rules (as flexible, strict), while the **VacationRentalPolicy** table links each vacation rental to specific cancellation policies. This separation allows multiple vacation rentals to share the same cancellation policy, avoiding duplication and ensuring flexibility in assigning policies.

1. Roles:

- **Guests**: Follow cancellation policies before/after booking.
- **Hosts**: Set and manage cancellation policies for their listings.
- **System Administrators**: Manage predefined policies and oversee updates.

2. Actions:

- **Guests**: Review and comply with cancellation terms.
- **Hosts**: Create or update policies for their listings.
- **System Administrators**: Define, update, and remove policy types.

3. Data:

- **PolicyID**: Unique identifier for each cancellation policy, automatically incremented.
- **PolicyName**: The name of the cancellation policy (Flexible, moderate, strict, etc.)
- **Description:** Text description providing details about the policy, such as refund conditions and applicable timelines.

4. Relationships:

- **VacationRental - CancellationPolicy**: Each vacation rental can be associated with a specific cancellation policy (one-to-many).
- **Booking - CancellationPolicy**: Booking is linked to the applicable cancellation policy.

VacationRentalPolicy Table

```
CREATE TABLE VacationRentalPolicy (
    VacationRentalID INT NOT NULL,
    PolicyID INT NOT NULL,
    PRIMARY KEY (VacationRentalID, PolicyID),
    FOREIGN KEY (VacationRentalID) REFERENCES VacationRental(VacationRentalID),
    FOREIGN KEY (PolicyID) REFERENCES CancellationPolicy(PolicyID)
);
```

```
INSERT INTO VacationRentalPolicy (VacationRentalID, PolicyID)
VALUES
(1, 1),
(2, 2),
(3, 3),
(4, 4),
(5, 5),
(6, 6),
(7, 7),
(8, 8),
(9, 9),
(10, 10),
(11, 11),
(12, 12),
(13, 13),
(14, 14),
(15, 15),
(16, 16),
(17, 17),
(18, 18),
(19, 19),
(20, 20);
```

VacationRentalID	PolicyID
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20

1. Roles:

- **Guests**: Follow rental-specific policies during their stay.
- **Hosts**: Define property rules and conditions.
- **System Administrators**: Set general policies, ensuring they are followed.

2. Actions:

- **Guests:** Review and agree to rental policies when booking.
- **Hosts:** Create, manage or update policies for their properties.
- **System Administrators:** Define platform policies and assist hosts.

3. Data:

- **VacationRentalID:** The unique identifier for the vacation rental property. It links this policy to a specific vacation rental.
- **PolicyID:** The unique identifier for the policy. It links a vacation rental to a specific policy, such as a cancellation policy.
- **PRIMARY KEY (VacationRentalID, PolicyID):** This composite primary key ensures that each vacation rental is uniquely associated with a specific policy.

4. Relationships:

- **VacationRental - VacationRentalPolicy:** A vacation rental can be linked to multiple policies (one-to-many).
- **Policy - VacationRentalPolicy:** A policy can be applied to multiple vacation rentals (one-to-many).

City Table

The City table stores general information about cities, while the Location table links specific vacation rental addresses to a city. This separation allows multiple locations within the same city, providing flexibility and avoiding data duplication.

```sql
CREATE TABLE City (
    CityID INT AUTO_INCREMENT PRIMARY KEY,
    CityName VARCHAR(100),
    Country VARCHAR(100)
);
```

```sql
INSERT INTO City (CityName, Country)
VALUES
    ('Berlin', 'Germany'),
    ('Stockholm', 'Sweden'),
    ('Paris', 'France'),
    ('Rome', 'Italy'),
    ('Seoul', 'South Korea'),
    ('Tokyo', 'Japan'),
    ('Gothenburg', 'Sweden'),
    ('Munich', 'Germany'),
    ('Lyon', 'France'),
    ('Milan', 'Italy'),
    ('Cairo', 'Egypt'),
    ('London', 'United Kingdom'),
    ('Sydney', 'Australia'),
    ('New York', 'USA'),
    ('Toronto', 'Canada'),
    ('Zurich', 'Switzerland'),
    ('Malmo', 'Sweden'),
    ('Hamburg', 'Germany'),
    ('Florence', 'Italy'),
```

CityID	CityName	Country
1	Berlin	Germany
2	Stockholm	Sweden
3	Paris	France
4	Rome	Italy
5	Seoul	South Korea
6	Tokyo	Japan

1. Roles:

- **Guests**: View cities to find available rentals.
- **Hosts**: Ensure city information is correct for their listings.
- **System Administrators**: Manage and update city records.

2. Actions:

- **Guests**: Search for rentals by city.
- **Hosts**: Add or verify city names and countries for their listings.
- **System Administrators**: Ensure unique and accurate city entries.

3. Data:

- **CityID**: Unique identifier for each city.
- **CityName**: Name of the city (Berlin, Paris, etc.).
- **Country**: Location of country.

4. Relationships:

- **City - Location**: Each city can have multiple linked locations (one-to-many).

Location Table

```sql
CREATE TABLE Location (
    LocationID INT AUTO_INCREMENT PRIMARY KEY,
    CityID INT,
    Country VARCHAR(100),
    PartOfCity VARCHAR(100),
    Address VARCHAR(255),
    PhoneNumber VARCHAR(20),
    Email VARCHAR(255),
    FOREIGN KEY (CityID) REFERENCES City(CityID)
);
```

```sql
INSERT INTO Location (CityID, Country, PartOfCity, Address, PhoneNumber, Email)
VALUES
    (1, 'Germany', 'Mitte', 'Alexanderplatz 1, 10178 Berlin', '49123456789', 'info@berlin.de'),
    (2, 'Sweden', 'Södermalm', 'Götgatan 100, 11862 Stockholm', '46701234567', 'info@stockholm.se'),
    (3, 'France', 'Montmartre', 'Rue Lepic 18, 75018 Paris', '33123456789', 'info@paris.fr'),
    (4, 'Italy', 'Trastevere', 'Via della Scala 12, 00153 Rome', '390123456789', 'info@rome.it'),
    (5, 'South Korea', 'Gangnam-gu', 'Teheran-ro 100, 06134 Seoul', '821012345678', 'info@seoul.kr'),
    (6, 'Japan', 'Shibuya', 'Harajuku 1-23-45, 150-0001 Tokyo', '819012345678', 'info@tokyo.jp'),
    (7, 'Sweden', 'Vasastan', 'Sveavägen 50, 11359 Gothenburg', '46709876543', 'info@gothenburg.se'),
    (8, 'Germany', 'Altstadt-Lehel', 'Marienplatz 1, 80331 Munich', '4915123456789', 'info@munich.de'),
    (9, 'France', 'La Croix-Rousse', 'Rue de la République 50, 69001 Lyon', '33112233445', 'info@lyon.fr'),
    (10, 'Italy', 'Brera', 'Via Fiori Chiari 20, 20121 Milan', '390223344556', 'info@milan.it'),
    (11, 'Egypt', 'Zamalek', '26th of July St, Cairo', '201234567890', 'info@cairo.eg'),
    (12, 'United Kingdom', 'Westminster', '10 Downing St, London SW1A 2AA', '441234567890', 'info@london.co.uk
    (13, 'Australia', 'The Rocks', 'George St 140, 2000 Sydney', '611234567890', 'info@sydney.au'),
    (14, 'USA', 'Manhattan', '5th Avenue 350, New York, NY 10001', '11234567890', 'info@newyork.us'),
    (15, 'Canada', 'Downtown', 'Bay St 200, Toronto, ON M5J 2R8', '11234567891', 'info@toronto.ca'),
    (16, 'Switzerland', 'Altstadt', 'Bahnhofstrasse 1, 8001 Zurich', '41234567890', 'info@zurich.ch'),
    (17, 'Sweden', 'Gamla Staden', 'Södra Förstadsgatan 15, 21143 Malmo', '46709876544', 'info@malmo.se'),
    (18, 'Germany', 'HafenCity', 'Sandtorkai 30, 20457 Hamburg', '4915223456789', 'info@hamburg.de'),
    (19, 'Italy', 'Santa Maria Novella', 'Piazza di Santa Maria Novella, 50123 Florence', '390334455667', 'inf
    (20, 'France', 'Promenade des Anglais', 'Quai des États-Unis 50, 06300 Nice', '33122334455', 'info@nice.fr
```

LocationID	CityID	Country	PartOfCity	Address	PhoneNumber	Email
1	1	Germany	Mitte	Alexanderplatz 1, 10178 Berlin	49123456789	info@berlin.de
2	2	Sweden	Södermalm	Götgatan 100, 11862 Stockholm	46701234567	info@stockholn
3	3	France	Montmartre	Rue Lepic 18, 75018 Paris	33123456789	info@paris.fr
4	4	Italy	Trastevere	Via della Scala 12, 00153 Rome	390123456789	info@rome.it
5	5	South Korea	Gangnam-gu	Teheran-ro 100, 06134 Seoul	821012345678	info@seoul.kr

1. Roles:

- **Guests**: Search and view detailed rental location information.
- **Hosts**: Add or update specific location details for their properties.
- **System Administrators**: Oversee and ensure correct location entries.

2. Actions:

- **Guests**: Browse locations for vacation rentals.
- **Hosts**: Enter specific location details (address and contact).
- **System Administrators**: Link locations to cities and maintain data accuracy.

3. Data:

- **LocationID**: Unique identifier for each location.
- **CityID**: Foreign key linking the location to a city.
- **Country**: Country where the location is situated.
- **PartOfCity**: Specific district or area within a city (Mitte, Montmartre, etc.).
- **Address**: Full address of the rental property.
- **PhoneNumber**: Contact phone number for the location.
- **Email**: Contact email for inquiries.

4. Relationships:

- **Location - City**: Each location is linked to one city (many-to-one).

CustomerService Table

```sql
CREATE TABLE CustomerService (
    CustomerServiceID INT AUTO_INCREMENT PRIMARY KEY,
    BookingID INT NOT NULL,
    IssueDescription TEXT,
    Resolution TEXT,
    ContactMethod VARCHAR(255),
    ResolutionDate DATE,
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)
);
```

```sql
INSERT INTO CustomerService (BookingID, IssueDescription, Resolution, ContactMethod, ResolutionDate)
VALUES
    (1, 'Late check-in', 'Offered late check-in', 'Phone', '2024-09-02'),
    (2, 'WiFi not working', 'Fixed WiFi issue', 'Email', '2024-09-06'),
    (3, 'Room not clean', 'Sent cleaning staff', 'Chat', '2024-09-11'),
    (4, 'No hot water', 'Repaired hot water system', 'Phone', '2024-09-13'),
    (5, 'Key not working', 'Provided new key', 'Email', '2024-09-16'),
    (6, 'Noise complaint', 'Offered room change', 'Chat', '2024-09-19'),
    (7, 'Room too cold', 'Adjusted thermostat', 'Phone', '2024-09-21'),
    (8, 'TV not working', 'Replaced TV', 'Email', '2024-09-26'),
    (9, 'No parking space', 'Arranged alternative parking', 'Chat', '2024-09-29'),
    (10, 'No towels in room', 'Delivered towels', 'Phone', '2024-10-02'),
    (11, 'Noisy neighbors', 'Spoke to neighbors', 'Email', '2024-10-04'),
    (12, 'Internet slow', 'Upgraded WiFi', 'Chat', '2024-10-08'),
    (13, 'Broken chair', 'Replaced chair', 'Phone', '2024-10-11'),
    (14, 'Shower not draining', 'Unclogged drain', 'Email', '2024-10-13'),
    (15, 'No hot water', 'Repaired hot water system', 'Chat', '2024-10-17'),
    (16, 'Air conditioning not working', 'Fixed AC', 'Phone', '2024-10-20'),
    (17, 'No blankets', 'Delivered blankets', 'Email', '2024-10-23'),
    (18, 'Broken lock', 'Fixed lock', 'Chat', '2024-10-27'),
    (19, 'Water leak', 'Repaired leak', 'Phone', '2024-10-30'),
    (20, 'Loud construction noise', 'Offered room change', 'Email', '2024-11-03');
```

CustomerServiceID	BookingID	IssueDescription	Resolution	ContactMethod	ResolutionDate
1	1	Late check-in	Offered late check-in	Phone	2024-09-02
2	2	WiFi not working	Fixed WiFi issue	Email	2024-09-06
3	3	Room not clean	Sent cleaning staff	Chat	2024-09-11
4	4	No hot water	Repaired hot water system	Phone	2024-09-13
5	5	Key not working	Provided new key	Email	2024-09-16
6	6	Noise complaint	Offered room change	Chat	2024-09-19
7	7	Room too cold	Adjusted thermostat	Phone	2024-09-21

1. Roles:

- **Guests**: Can request assistance related to their bookings.
- **Hosts**: Request support regarding bookings or property issues.
- **Customer Service Representatives**: Provides support and resolve issues for both guests and hosts.
- **System Administrators**: Manage customer service requests and feedback.

2. Actions:

- **Guests & Hosts:** Submit service requests related to bookings.
- **Customer Service Representatives**: Escalate and resolve issues.
- **System Administrators**: Track service requests, improve service quality, and find solutions.

3. Data:

- **CustomerServiceID:** Unique identifier for each service request.
- **BookingID**: Links the request to the specific booking.
- **IssueDescription:** Detailed description of the problem or concern.
- **Resolution:** Type of resolution offered for the issue.
- **ContactMethod:** How the customer was contacted (phone, email.)
- **ResolutionDate**: Records the date the request was submitted.

4. Relationships:

- **CustomerService - Booking**: Each customer service entry is linked to a specific booking (BookingID).
- **Booking - CustomerService**: One booking can be associated with multiple customer service issues.

SocialNetwork Table

The **SocialNetwork** table stores information about social media platforms, including the name and URL of each network (e.g., Facebook, Twitter).
The **GuestSocialNetwork** table links guests to their social media profiles by referencing both the **Guest** and **SocialNetwork** tables. It allows guests to associate specific social media profiles (e.g., profile URL) with the social networks they use.

```sql
CREATE TABLE SocialNetwork (
    NetworkID INT AUTO_INCREMENT PRIMARY KEY,
    NetworkName VARCHAR(100),
    URL VARCHAR(255)
);
```

```sql
INSERT INTO SocialNetwork (NetworkName, URL)
VALUES
    ('Facebook', 'https://www.facebook.com'),
    ('Twitter', 'https://www.twitter.com'),
    ('Instagram', 'https://www.instagram.com'),
    ('LinkedIn', 'https://www.linkedin.com'),
    ('WhatsApp', 'https://www.whatsapp.com'),
    ('WeChat', 'https://www.wechat.com'),
    ('VK', 'https://www.vk.com'),
    ('Snapchat', 'https://www.snapchat.com'),
    ('TikTok', 'https://www.tiktok.com'),
    ('Pinterest', 'https://www.pinterest.com'),
    ('Reddit', 'https://www.reddit.com'),
    ('Tumblr', 'https://www.tumblr.com'),
    ('Flickr', 'https://www.flickr.com'),
    ('YouTube', 'https://www.youtube.com'),
    ('Vimeo', 'https://www.vimeo.com'),
    ('Discord', 'https://www.discord.com'),
    ('Telegram', 'https://www.telegram.org'),
    ('Signal', 'https://www.signal.org'),
    ('Baidu Tieba', 'https://tieba.baidu.com'),
    ('Douban', 'https://www.douban.com');
```

NetworkID	NetworkName	URL
1	Facebook	https://www.facebook.com
2	Twitter	https://www.twitter.com
3	Instagram	https://www.instagram.com
4	LinkedIn	https://www.linkedin.com
5	WhatsApp	https://www.whatsapp.com

1. Roles:

- **Guests and Hosts:** Use their social network accounts to link profiles or authenticate on the platform.
- **System Administrators:** Manage the supported social networks available for account linking or sharing purposes.

2. Actions:

- **Guests and Hosts:** Link their social media accounts to the platform, such as Facebook, Twitter, or Instagram.
- **System Administrators:** Add new social network platforms and manage their integration.

3. Data:

- **NetworkID:** Unique identifier for each social network.
- **NetworkName:** The name of the social network (Facebook, Instagram, etc.).
- **URL:** The base URL for the social network (www.facebook.com, etc.).

4. Relationships:

- **GuestSocialNetwork - SocialNetwork:** Guests and hosts can link their social media accounts, which are referenced from the social network table (many-to-one relationship).

GuestSocialNetwork Table

```sql
CREATE TABLE GuestSocialNetwork (
    GuestSocialNetworkID INT AUTO_INCREMENT PRIMARY KEY,
    GuestID INT NOT NULL,
    NetworkID INT NOT NULL,
    ProfileURL VARCHAR(255),
    FOREIGN KEY (GuestID) REFERENCES Guest(GuestID),
    FOREIGN KEY (NetworkID) REFERENCES SocialNetwork(NetworkID)
);
```

```sql
INSERT INTO GuestSocialNetwork (GuestID, NetworkID, ProfileURL)
VALUES
    (1, 1, 'https://www.facebook.com/hans.mueller'),
    (2, 2, 'https://www.twitter.com/emma.karlsson'),
    (3, 3, 'https://www.instagram.com/jean.dupont'),
    (4, 4, 'https://www.linkedin.com/in/giovanni.rossi'),
    (5, 5, 'https://www.whatsapp.com/seo.jun.lee'),
    (6, 6, 'https://www.wechat.com/sakura.tanaka'),
    (7, 7, 'https://www.vk.com/anna.svensson'),
    (8, 8, 'https://www.snapchat.com/friedrich.becker'),
    (9, 9, 'https://www.tiktok.com/pierre.martin'),
    (10, 10, 'https://www.pinterest.com/alessandro.bianchi'),
    (11, 11, 'https://www.reddit.com/user/ahmed.elsayed'),
    (12, 12, 'https://www.tumblr.com/john.smith'),
    (13, 13, 'https://www.flickr.com/olivia.brown'),
    (14, 14, 'https://www.youtube.com/michael.johnson'),
    (15, 15, 'https://www.vimeo.com/emma.williams'),
    (16, 16, 'https://www.discord.com/chloe.dubois'),
    (17, 17, 'https://www.telegram.org/mats.andersson'),
    (18, 18, 'https://www.signal.org/sophie.schmidt'),
    (19, 19, 'https://tieba.baidu.com/luca.bruni'),
    (20, 20, 'https://www.douban.com/isabelle.laurent');
```

GuestSocialNetworkID	GuestID	NetworkID	ProfileURL
1	1	1	https://www.facebook.com/hans.mueller
2	2	2	https://www.twitter.com/emma.karlsson
3	3	3	https://www.instagram.com/jean.dupont
4	4	4	https://www.linkedin.com/in/giovanni.rossi
5	5	5	https://www.whatsapp.com/seo.jun.lee

1. Roles:

- **Guests**: Link their social network profiles to their accounts.
- **System Administrators**: Ensure that guest profiles are properly linked to their social media accounts.

2. Actions:

- **Guests**: Add or update social network profile information.
- **System Administrators**: Maintain social network connections.

3. Data:

- **GuestSocialNetworkID**: Unique identifier for each guest's social network post.
- **GuestID**: Links the social network profile to the guest's account.
- **NetworkID**: Identifies the social network (Facebook, Twitter, etc.).
- **ProfileURL**: The URL to the guest's profile on the social network.

4. Relationships:

- **Guest - GuestSocialNetwork**: A guest can have multiple social network profiles linked to their account.
- **SocialNetwork - GuestSocialNetwork**: A social network (Facebook, etc.) can be associated with multiple guests.

Amenity Table

The VacationRentalAmenity Table is a junction (associative) table that connects the VacationRental table and the Amenity table. It handles the many-to-many relationship between vacation rentals and amenities.

Each row in this table represents one amenity being available at one specific vacation rental.

```sql
CREATE TABLE Amenity (
    AmenityID INT AUTO_INCREMENT PRIMARY KEY,
    AmenityName VARCHAR(100),
    Description TEXT
);
```

```sql
INSERT INTO Amenity (AmenityName, Description)
VALUES
    ('WiFi', 'High-speed internet access'),
    ('Air Conditioning', 'Room equipped with air conditioning'),
    ('Heating', 'Central heating system'),
    ('Parking', 'Free parking space available'),
    ('Swimming Pool', 'Access to a swimming pool'),
    ('Kitchen', 'Fully equipped kitchen'),
    ('Laundry', 'Laundry facilities available'),
    ('Gym', 'Access to a gym'),
    ('Pet Friendly', 'Pets are allowed'),
    ('Breakfast', 'Breakfast included in the stay'),
    ('Balcony', 'Room with a balcony'),
    ('Terrace', 'Access to a terrace'),
    ('Sea View', 'Room with a view of the sea'),
    ('Fireplace', 'Room equipped with a fireplace'),
    ('TV', 'Television available in the room'),
    ('Garden', 'Access to a garden'),
    ('Hot Tub', 'Hot tub available'),
    ('Sauna', 'Access to a sauna'),
    ('Barbecue', 'Barbecue facilities available'),
    ('Bicycle Rental', 'Bicycles available for rent');
```

AmenityID	AmenityName	Description
1	WiFi	High-speed internet access
2	Air Conditioning	Room equipped with air conditioning
3	Heating	Central heating system
4	Parking	Free parking space available
5	Swimming Pool	Access to a swimming pool

1. Roles:

- **Guests**: View available amenities at vacation rentals.
- **Hosts**: Define and list amenities provided in their properties.
- **System Administrators**: Oversee amenity listings across the platform.

2. Actions:

- **Guests**: Check amenities while booking a property.
- **Hosts**: Add, update, or remove amenities for their properties.

3. Data:

- **AmenityID**: Unique identifier for each amenity (Primary Key).
- **AmenityName:** Wi-Fi, Pool, Parking, etc.
- **Description**: Details or extra information about the amenity.

4. Relationships:

- **No Direct Relationships:** The Amenity table does not contain any foreign keys and therefore does not have direct relationships with other tables.
- **Lookup Table:** It serves as a reference table, where each amenity (Wi-Fi, Pool, etc.) can be referenced elsewhere in the system, such as in a junction table that links properties to amenities.

VacationRentalAmenity Table

```sql
CREATE TABLE VacationRentalAmenity (
    VacationRentalID INT NOT NULL,
    AmenityID INT NOT NULL,
    PRIMARY KEY (VacationRentalID, AmenityID),
    FOREIGN KEY (VacationRentalID) REFERENCES VacationRental(VacationRentalID),
    FOREIGN KEY (AmenityID) REFERENCES Amenity(AmenityID)
);
```

```sql
INSERT INTO VacationRentalAmenity (VacationRentalID, AmenityID)
VALUES
    (1, 1),  -- WiFi for VacationRentalID 1
    (2, 2),  -- Air Conditioning for VacationRentalID 2
    (3, 3),  -- Heating for VacationRentalID 3
    (4, 4),  -- Parking for VacationRentalID 4
    (5, 5),  -- Swimming Pool for VacationRentalID 5
    (6, 6),  -- Kitchen for VacationRentalID 6
    (7, 7),  -- Laundry for VacationRentalID 7
    (8, 8),  -- Gym for VacationRentalID 8
    (9, 9),  -- Pet Friendly for VacationRentalID 9
    (10, 10), -- Breakfast for VacationRentalID 10
    (11, 11), -- Balcony for VacationRentalID 11
    (12, 12), -- Terrace for VacationRentalID 12
    (13, 13), -- Sea View for VacationRentalID 13
    (14, 14), -- Fireplace for VacationRentalID 14
    (15, 15), -- TV for VacationRentalID 15
    (16, 16), -- Garden for VacationRentalID 16
    (17, 17), -- Hot Tub for VacationRentalID 17
    (18, 18), -- Sauna for VacationRentalID 18
    (19, 19), -- Barbecue for VacationRentalID 19
    (20, 20); -- Bicycle Rental for VacationRentalID 20
```

VacationRentalID	AmenityID
1	1
2	2
3	3
4	4
5	5

1. Roles:

- **Guests:** View available amenities when booking a vacation rental.
- **Hosts:** Define and manage the amenities offered at their vacation rentals.
- **System Administrators:** Oversee and ensure the proper linking of amenities to vacation rentals.

2. Actions:

- **Guests:** View amenities during the booking process.
- **Hosts:** Assign or update amenities for each rental property.
- **System Administrators:** Maintain the database structure, ensure accurate linking of rentals and amenities.

3. Data:

- **VacationRentalID:** Unique identifier linking a vacation rental to specific amenities.
- **AmenityID:** Unique identifier for each amenity (Wi-Fi, Pool, etc.).

4. Relationships:

- **VacationRental - VacationRentalAmenity:** Each vacation rental can have multiple amenities.
- **Amenity - VacationRentalAmenity:** Each amenity can be linked to multiple vacation rentals (many-to-many relationship).

Room Table

The **Room** table represents individual units or spaces within a vacation rental, such as a suite or deluxe room, with specific details like room type and nightly price.

The **VacationRental** table on the other hand refers to the entire property available for rent, like a house or apartment, and includes general property details such as location, maximum guests, and amenities available for the whole rental.

```
CREATE TABLE Room (
    RoomID INT AUTO_INCREMENT PRIMARY KEY,
    VacationRentalID INT NOT NULL,
    RoomType VARCHAR(100),
    PricePerNight FLOAT,
    AvailableFrom DATE,
    AvailableTo DATE,
    FOREIGN KEY (VacationRentalID) REFERENCES VacationRental(VacationRentalID)
);
```

```
INSERT INTO Room (VacationRentalID, RoomType, PricePerNight, AvailableFrom, AvailableTo
VALUES
    (1, 'Deluxe', 150.00, '2024-01-01', '2024-03-31'),
    (2, 'Standard', 120.00, '2024-04-01', '2024-06-30'),
    (3, 'Deluxe', 180.00, '2024-07-01', '2024-09-30'),
    (4, 'Suite', 250.00, '2024-10-01', '2024-12-31'),
    (5, 'Standard', 90.00, '2024-02-01', '2024-05-31'),
    (6, 'Deluxe', 140.00, '2024-06-01', '2024-09-30'),
    (7, 'Cottage', 130.00, '2024-04-01', '2024-08-31'),
    (8, 'Standard', 100.00, '2024-05-01', '2024-10-31'),
    (9, 'Deluxe', 160.00, '2024-03-01', '2024-06-30'),
    (10, 'Standard', 120.00, '2024-07-01', '2024-12-31'),
    (11, 'Economy', 85.00, '2024-01-15', '2024-04-15'),
    (12, 'Standard', 200.00, '2024-05-01', '2024-08-31'),
    (13, 'Penthouse', 300.00, '2024-06-01', '2024-11-30'),
    (14, 'Loft', 160.00, '2024-03-01', '2024-07-31'),
    (15, 'Economy', 100.00, '2024-02-01', '2024-05-31'),
    (16, 'Chalet', 220.00, '2024-06-01', '2024-09-30'),
    (17, 'Townhouse', 130.00, '2024-04-01', '2024-07-31'),
    (18, 'Studio', 70.00, '2024-03-01', '2024-10-31'),
    (19, 'Suite', 240.00, '2024-05-01', '2024-09-30'),
    (20, 'Standard', 105.00, '2024-07-01', '2024-12-31');
```

RoomID	VacationRentalID	RoomType	PricePerNight	AvailableFrom	AvailableTo
1	1	Deluxe	150	2024-01-01	2024-03-31
2	2	Standard	120	2024-04-01	2024-06-30
3	3	Deluxe	180	2024-07-01	2024-09-30
4	4	Suite	250	2024-10-01	2024-12-31
5	5	Standard	90	2024-02-01	2024-05-31

1. Roles:

- **Guests:** View room availability and pricing when searching for vacation rentals.
- **Hosts:** Define room types, pricing, and availability for their vacation rental properties.
- **System Administrators:** Ensure data integrity and maintain the association between vacation rentals and rooms.

2. Actions:

- **Guests:** Search for available rooms, price and dates.
- **Hosts:** Add, update, or remove room details including availability, pricing, and room type.
- **System Administrators:** Oversee room data, ensuring it is accurate and linked to the correct vacation rentals.

3. Data:

- **RoomID:** Unique identifier for each room.
- **VacationRentalID:** Links the room to a specific vacation rental.
- **RoomType:** Specifies the type of room (e.g., single, double, suite).
- **PricePerNight:** The nightly rate for the room.
- **AvailableFrom & AvailableTo:** Specifies the availability date range of the room.

4. Relationships:

- **VacationRental-Room:** Each room is associated with a specific vacation rental, and a vacation rental can have multiple rooms.

VacationRental Table

```sql
CREATE TABLE VacationRental (
    VacationRentalID INT AUTO_INCREMENT PRIMARY KEY,
    HostID INT NOT NULL,
    LocationID INT NOT NULL,
    PropertyType VARCHAR(100),
    Description TEXT,
    MaxGuests INT,
    RatePerPerson FLOAT,
    OwnBathroom BOOLEAN,
    DogFriendly BOOLEAN,
    FreeParking BOOLEAN,
    NumberOfBeds INT,
    CalendarAvailability TEXT,
    ProximityToBeach VARCHAR(255),
    ProximityToShops VARCHAR(255),
    ProximityToSightSeeing VARCHAR(255),
    FOREIGN KEY (HostID) REFERENCES Host(HostID),
    FOREIGN KEY (LocationID) REFERENCES Location(LocationID)
);
```

```sql
INSERT INTO VacationRental (
    HostID, LocationID, PropertyType, Description, MaxGuests, RatePerPerson, OwnBathroom,
    DogFriendly, FreeParking, NumberOfBeds, CalendarAvailability, ProximityToBeach,
    ProximityToShops, ProximityToSightSeeing)
VALUES
  (1, 1, 'Apartment', 'Beautiful luxury apartment in the city center.', 4, 120.00, 1, 1, 1, 2, 'Available', '500m', '200m', '300m'),
  (2, 2, 'House', 'Spacious family house with a garden.', 6, 150.00, 1, 1, 1, 3, 'Available', '1km', '500m', '700m'),
  (3, 3, 'Apartment', 'Cozy apartment near the river.', 2, 80.00, 1, 0, 1, 1, 'Available', '300m', '100m', '200m'),
  (4, 4, 'Villa', 'Luxury villa with ocean views.', 8, 250.00, 1, 1, 1, 4, 'Available', '50m', '1km', '500m'),
  (5, 5, 'Condo', 'Modern condo in a high-rise building.', 3, 90.00, 1, 0, 1, 1, 'Available', '800m', '200m', '400m'),
  (6, 6, 'Cottage', 'Charming cottage in the countryside.', 5, 130.00, 1, 1, 1, 2, 'Available', '10km', '5km', '6km'),
  (7, 7, 'Townhouse', 'Stylish townhouse in a vibrant neighborhood.', 4, 140.00, 1, 1, 1, 2, 'Available', '1km', '500m', '800m'),
  (8, 8, 'Penthouse', 'Top-floor penthouse with panoramic city views.', 4, 300.00, 1, 1, 1, 2, 'Available', '200m', '100m', '300m'),
  (9, 9, 'Villa', 'Exclusive villa with private pool.', 10, 400.00, 1, 1, 1, 5, 'Available', '50m', '300m', '400m'),
  (10, 10, 'Apartment', 'Affordable apartment close to the city center.', 3, 70.00, 1, 0, 1, 1, 'Available', '600m', '400m', '500m'),
  (11, 11, 'Loft', 'Stylish loft in the heart of the city.', 2, 150.00, 1, 0, 1, 1, 'Available', '400m', '150m', '300m'),
  (12, 12, 'Chalet', 'Cozy mountain chalet with fireplace.', 6, 180.00, 1, 1, 1, 3, 'Available', '1km', '600m', '800m'),
  (13, 13, 'Bungalow', 'Beachfront bungalow in a tropical setting.', 4, 200.00, 1, 1, 1, 2, 'Available', '50m', '100m', '200m'),
  (14, 14, 'Cabin', 'Secluded cabin in the woods.', 5, 120.00, 1, 1, 1, 2, 'Available', '5km', '3km', '4km'),
  (15, 15, 'Studio', 'Compact studio in a quiet area.', 2, 60.00, 1, 0, 1, 1, 'Available', '500m', '300m', '400m'),
  (16, 16, 'Mansion', 'Luxurious mansion with sprawling gardens.', 12, 500.00, 1, 1, 1, 6, 'Available', '1km', '500m', '700m'),
  (17, 17, 'Townhouse', 'Spacious townhouse perfect for families.', 6, 160.00, 1, 1, 1, 3, 'Available', '700m', '200m', '300m'),
  (18, 18, 'Villa', 'Exclusive villa with private beach access.', 8, 350.00, 1, 1, 1, 4, 'Available', '0m', '100m', '200m'),
  (19, 19, 'Apartment', 'Cozy apartment for budget travelers.', 2, 50.00, 1, 0, 1, 1, 'Available', '800m', '400m', '600m'),
  (20, 20, 'Cottage', 'Charming cottage with garden.', 5, 140.00, 1, 1, 1, 2, 'Available', '2km', '500m', '700m');
```

VacationRentalID	HostID	LocationID	PropertyType	Description	MaxGuests	RatePerPerson	OwnBathroom	DogFrien
1	1	1	Apartment	Beautiful luxury apartment in the city center.	4	120	1	1
2	2	2	House	Spacious family house with a garden.	6	150	1	1
3	3	3	Apartment	Cozy apartment near the river.	2	80	1	0
4	4	4	Villa	Luxury villa with ocean views.	8	250	1	1
5	5	5	Condo	Modern condo in a high-rise building.	3	90	1	0
6	6	6	Cottage	Charming cottage in the countryside.	5	130	1	1
7	7	7	Townhouse	Stylish townhouse in a vibrant neighborhood.	4	140	1	1

1. Roles:

- **Guests**: Browse and book vacation rentals.
- **Hosts**: Manage the details of their vacation rentals.
- **System Administrators**: Oversee and maintain the accuracy of vacation rental data.

2. Actions:

- **Guests**: Search and view rental details, book rentals based on availability.
- **Hosts**: Create, update, or remove their vacation rentals.
- **System Administrators**: Ensure rental data is up to date, monitor system-wide records.

3. Data:

- **VacationRentalID**: Unique identifier for each rental property.
- **HostID**: Links the rental to the host.
- **LocationID**: Links the rental to a specific location.
- **PropertyType**: The type of rental (loft, villa, etc.).
- **MaxGuests**: Allowed maximum number of guests.
- **RatePerPerson**: Price charged per person.
- **CalendarAvailability**: Availability of the rental property.
- **Proximity Details**: Distance to beach, shops, sightseeing.
- **Other Features**: Own bathroom, dog-friendly, free parking, number of beds.

4. Relationships:

- **Host -VacationRental**: Each vacation rental is linked to a host (via HostID).
- **Location - VacationRental**: Each rental is connected to a specific location (via LocationID).

Notification Table

```sql
CREATE TABLE Notification (
    NotificationID INT AUTO_INCREMENT PRIMARY KEY,
    GuestID INT NOT NULL,
    Content TEXT,
    Timestamp DATETIME,
    FOREIGN KEY (GuestID) REFERENCES Guest(GuestID)
);
```

```sql
INSERT INTO Notification (GuestID, Content, Timestamp)
VALUES
    (1, 'Welcome to our service!', '2024-09-01 08:10:00'),
    (2, 'Your booking has been confirmed.', '2024-09-01 09:15:00'),
    (3, 'Payment successful.', '2024-09-01 10:20:00'),
    (4, 'Your stay in Rome is coming up.', '2024-09-01 11:25:00'),
    (5, 'Check-in available for your booking.', '2024-09-01 12:30:00'),
    (6, 'Reminder: Leave a review for your recent stay.', '2024-09-01 13:35:00'),
    (7, 'New message from your host.', '2024-09-01 14:40:00'),
    (8, 'Special offer for your next stay.', '2024-09-01 15:45:00'),
    (9, 'Booking cancellation successful.', '2024-09-01 16:50:00'),
    (10, 'Your refund has been processed.', '2024-09-01 17:55:00'),
    (11, 'New property added in your favorite city.', '2024-09-01 18:00:00'),
    (12, 'Update your profile to get better recommendations.', '2024-09-01 19:05:00'),
    (13, 'New loyalty points added to your account.', '2024-09-01 20:10:00'),
    (14, 'Last chance to book your dream vacation.', '2024-09-01 21:15:00'),
    (15, 'Your stay in New York is coming up.', '2024-09-01 22:20:00'),
    (16, 'Your booking for Zurich has been confirmed.', '2024-09-01 23:25:00'),
    (17, 'Check-out reminder for your stay.', '2024-09-02 00:30:00'),
    (18, 'Thank you for staying with us!', '2024-09-02 01:35:00'),
    (19, 'Special promotion: 20% off your next booking.', '2024-09-02 02:40:00'),
    (20, 'Your profile has been updated successfully.', '2024-09-02 03:45:00');
```

NotificationID	GuestID	Content	Timestamp
1	1	Welcome to our service!	2024-09-01 08:10:00
2	2	Your booking has been confirmed.	2024-09-01 09:15:00
3	3	Payment successful.	2024-09-01 10:20:00
4	4	Your stay in Rome is coming up.	2024-09-01 11:25:00
5	5	Check-in available for your booking.	2024-09-01 12:30:00

1. Roles:

- **Guests**: Receive and view notifications.
- **System Administrators**: Manage notifications sent to guests.

2. Actions:

- **Guests**: View and respond to notifications.
- **System Administrators**: Create and send notifications to guests.

3. Data:

- **NotificationID**: Unique identifier for each notification.
- **GuestID**: Links the notification to a specific guest.
- **Content**: The message or content of the notification.
- **Timestamp**: The date and time when the notification was created.

4. Relationships:

- **Guest - Notification**: Each notification is linked to one guest, but a guest can have multiple notifications.

Promotion Table

```sql
CREATE TABLE Promotion (
    PromotionID INT AUTO_INCREMENT PRIMARY KEY,
    VacationRentalID INT NOT NULL,
    DiscountPercentage FLOAT,
    StartDate DATE,
    EndDate DATE,
    FOREIGN KEY (VacationRentalID) REFERENCES VacationRental(VacationRentalID)
);
```

```sql
INSERT INTO Promotion (VacationRentalID, DiscountPercentage, StartDate, EndDate)
VALUES
    (1, 10.0, '2024-09-01', '2024-09-07'),  -- Hans Müller
    (2, 15.0, '2024-09-05', '2024-09-10'),  -- Emma Karlsson
    (3, 12.5, '2024-09-10', '2024-09-15'),  -- Jean Dupont
    (4, 20.0, '2024-09-12', '2024-09-18'),  -- Giovanni Rossi
    (5, 8.0, '2024-09-15', '2024-09-20'),   -- Seo-jun Lee
    (6, 10.0, '2024-09-18', '2024-09-22'),  -- Sakura Tanaka
    (7, 7.5, '2024-09-20', '2024-09-25'),   -- Anna Svensson
    (8, 18.0, '2024-09-25', '2024-09-30'),  -- Friedrich Becker
    (9, 12.0, '2024-09-28', '2024-10-03'),  -- Pierre Martin
    (10, 14.0, '2024-10-01', '2024-10-05'), -- Alessandro Bianchi
    (11, 9.5, '2024-10-03', '2024-10-08'),  -- Ahmed El-Sayed
    (12, 11.0, '2024-10-07', '2024-10-12'), -- John Smith
    (13, 25.0, '2024-10-10', '2024-10-15'), -- Olivia Brown
    (14, 19.0, '2024-10-12', '2024-10-17'), -- Michael Johnson
    (15, 15.0, '2024-10-15', '2024-10-20'), -- Emma Williams
    (16, 17.5, '2024-10-18', '2024-10-22'), -- Chloe Dubois
    (17, 20.0, '2024-10-20', '2024-10-25'), -- Mats Andersson
    (18, 5.0, '2024-10-25', '2024-10-30'),  -- Sophie Schmidt
    (19, 22.0, '2024-10-28', '2024-11-02'), -- Luca Bruni
    (20, 10.0, '2024-11-01', '2024-11-05'); -- Isabelle Laurent
```

PromotionID	VacationRentalID	DiscountPercentage	StartDate	EndDate
1	1	10	2024-09-01	2024-09-07
2	2	15	2024-09-05	2024-09-10
3	3	12.5	2024-09-10	2024-09-15
4	4	20	2024-09-12	2024-09-18
5	5	8	2024-09-15	2024-09-20

The **Promotion** table is linked to **VacationRental** and manages discounts and special offers which affects the price guests pay indirectly. It influences the rental price without being directly tied to the payments table.

1. Roles:

- **Guests**: View promotions for properties they are interested in.
- **Hosts**: Create and manage promotions for their vacation rentals.
- **System Administrators**: Oversee and maintain promotional campaigns.

2. Actions:

- **Guests**: View and apply promotions when booking rentals.
- **Hosts**: Create and update promotions for their properties.
- **System Administrators**: Ensure proper application and visibility of promotions.

3. Data:

- **PromotionID**: Unique identifier for each promotion.
- **VacationRentalID**: Links the promotion to a specific vacation rental.
- **DiscountPercentage**: The discount amount offered through the promotion.
- **StartDate** & **EndDate**: The period during which the promotion is active.

4. Relationships:

- **VacationRental - Promotion**: Each promotion is tied to one vacation rental, but a rental can have multiple promotions at different times.

LoginHistory Table

```sql
CREATE TABLE LoginHistory (
    LoginID INT AUTO_INCREMENT PRIMARY KEY,
    GuestID INT NOT NULL,
    LoginTimestamp DATETIME,
    IPAddress VARCHAR(45),
    FOREIGN KEY (GuestID) REFERENCES Guest(GuestID)
);
```

```sql
INSERT INTO LoginHistory (GuestID, LoginTimestamp, IPAddress)
VALUES
    (1, '2024-09-01 08:00:00', '192.168.1.1'),
    (2, '2024-09-01 09:00:00', '192.168.1.2'),
    (3, '2024-09-01 10:00:00', '192.168.1.3'),
    (4, '2024-09-01 11:00:00', '192.168.1.4'),
    (5, '2024-09-01 12:00:00', '192.168.1.5'),
    (6, '2024-09-01 13:00:00', '192.168.1.6'),
    (7, '2024-09-01 14:00:00', '192.168.1.7'),
    (8, '2024-09-01 15:00:00', '192.168.1.8'),
    (9, '2024-09-01 16:00:00', '192.168.1.9'),
    (10, '2024-09-01 17:00:00', '192.168.1.10'),
    (11, '2024-09-01 18:00:00', '192.168.1.11'),
    (12, '2024-09-01 19:00:00', '192.168.1.12'),
    (13, '2024-09-01 20:00:00', '192.168.1.13'),
    (14, '2024-09-01 21:00:00', '192.168.1.14'),
    (15, '2024-09-01 22:00:00', '192.168.1.15'),
    (16, '2024-09-01 23:00:00', '192.168.1.16'),
    (17, '2024-09-02 00:00:00', '192.168.1.17'),
    (18, '2024-09-02 01:00:00', '192.168.1.18'),
    (19, '2024-09-02 02:00:00', '192.168.1.19'),
    (20, '2024-09-02 03:00:00', '192.168.1.20');
```

LoginID	GuestID	LoginTimestamp	IPAddress
1	1	2024-09-01 08:00:00	192.168.1.1
2	2	2024-09-01 09:00:00	192.168.1.2
3	3	2024-09-01 10:00:00	192.168.1.3
4	4	2024-09-01 11:00:00	192.168.1.4
5	5	2024-09-01 12:00:00	192.168.1.5

1. Roles:

- **Guests/Hosts:** Their login activity is tracked for security and monitoring purposes.
- **System Administrators:** Monitor user activity and maintain security measures.

2. Actions:

- **Guests/Hosts:** Log in to the system.
- **System Administrators:** Track and review login activity, ensure system security.

3. Data:

- **LoginHistoryID:** Unique identifier for each login session.
- **UserID:** Refers to the guest or host who logged in.
- **LoginTime:** Timestamp indicating when the user logged in.
- **LogoutTime:** Timestamp indicating when the user logged out.
- **IPAddress:** The IP address used during login.

4. Relationships:

- **User - LoginHistory:** Each user can have multiple login entries recorded.

Event Table

```sql
CREATE TABLE Event (
    EventID INT AUTO_INCREMENT PRIMARY KEY,
    BookingID INT NOT NULL,
    EventName VARCHAR(255),
    StartDate DATE,
    EndDate DATE,
    Description TEXT,
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)
);
```

```sql
INSERT INTO Event (BookingID, EventName, StartDate, EndDate, Description)
VALUES
    (1, 'Berlin Marathon', '2024-09-08', '2024-09-10', 'Annual marathon event'),
    (2, 'Stockholm Music Festival', '2024-09-11', '2024-09-13', 'Music festival in Stockholm'),
    (3, 'Paris Fashion Week', '2024-09-14', '2024-09-20', 'Fashion event in Paris'),
    (4, 'Rome Film Festival', '2024-09-21', '2024-09-25', 'Film festival in Rome'),
    (5, 'Seoul Food Festival', '2024-09-26', '2024-09-28', 'Food festival in Seoul'),
    (6, 'Tokyo Game Show', '2024-09-29', '2024-10-01', 'Gaming event in Tokyo'),
    (7, 'Gothenburg Book Fair', '2024-10-02', '2024-10-05', 'Book fair in Gothenburg'),
    (8, 'Munich Oktoberfest', '2024-10-06', '2024-10-10', 'Beer festival in Munich'),
    (9, 'Lyon Lights Festival', '2024-10-11', '2024-10-13', 'Light festival in Lyon'),
    (10, 'Milan Fashion Week', '2024-10-14', '2024-10-18', 'Fashion event in Milan'),
    (11, 'Cairo Film Festival', '2024-10-19', '2024-10-22', 'Film festival in Cairo'),
    (12, 'London Literature Festival', '2024-10-23', '2024-10-27', 'Literature festival in London'),
    (13, 'Sydney Opera Festival', '2024-10-28', '2024-11-01', 'Opera festival in Sydney'),
    (14, 'New York Comic Con', '2024-11-02', '2024-11-05', 'Comic convention in New York'),
    (15, 'Toronto Film Festival', '2024-11-06', '2024-11-10', 'Film festival in Toronto'),
    (16, 'Zurich Art Festival', '2024-11-11', '2024-11-14', 'Art festival in Zurich'),
    (17, 'Malmo Music Festival', '2024-11-15', '2024-11-17', 'Music festival in Malmo'),
    (18, 'Hamburg Christmas Market', '2024-11-18', '2024-11-20', 'Christmas market in Hamburg'),
    (19, 'Florence Food Festival', '2024-11-21', '2024-11-24', 'Food festival in Florence'),
    (20, 'Nice Jazz Festival', '2024-11-25', '2024-11-27', 'Jazz festival in Nice');
```

EventID	BookingID	EventName	StartDate	EndDate	Description
1	1	Berlin Marathon	2024-09-08	2024-09-10	Annual marathon event
2	2	Stockholm Music Festival	2024-09-11	2024-09-13	Music festival in Stockholm
3	3	Paris Fashion Week	2024-09-14	2024-09-20	Fashion event in Paris
4	4	Rome Film Festival	2024-09-21	2024-09-25	Film festival in Rome
5	5	Seoul Food Festival	2024-09-26	2024-09-28	Food festival in Seoul
6	6	Tokyo Game Show	2024-09-29	2024-10-01	Gaming event in Tokyo

1. Roles:

- **Guests**: Attend or view information about events linked to their bookings.
- **Hosts**: Can view any events related to their rental bookings.
- **System Administrators**: Link events to relevant bookings or update event details.

2. Actions:

- **Guests & Hosts**: View event details linked to their bookings.
- **System Administrators**: Ensure event and booking relationships are maintained.

3. Data:

- **EventID**: Unique identifier for each event.
- **BookingID**: Links the event to a specific booking.
- **EventName**: The name or title of the event.
- **StartDate & EndDate**: Of an event.
- **Description**: Additional details about the event.

4. Relationships:

- **Booking-Event**: One booking can have multiple events associated with it (one-to-many relationship).

Test Cases

Test cases are queries that verify if the data and design are correct, using joins to extract data from three or more tables.

```sql
SELECT
    g.Name AS GuestName,
    b.BookingDate,
    b.CheckInDate,
    b.CheckOutDate,
    b.TotalPrice,
    vr.PropertyType,
    h.HostID,
    h.Rating AS HostRating
FROM Booking b
JOIN Guest g ON b.GuestID = g.GuestID
JOIN Room r ON b.RoomID = r.RoomID
JOIN VacationRental vr ON r.VacationRentalID = vr.VacationRentalID
JOIN Host h ON vr.HostID = h.HostID
ORDER BY b.BookingDate DESC;
```

This query joins 5 tables: Booking, Guest, Room, VacationRental, and Host, retrieving guest name, booking details, property type, and host rating using inner joins on related foreign keys.

GuestName	BookingDate	CheckInDate	CheckOutDate	TotalPrice	PropertyType	HostID	HostRating
Hans Müller	2024-09-06 01:12:57	2024-09-01	2024-09-07	900	Apartment	1	4.5
Emma Karlsson	2024-09-06 01:12:57	2024-09-05	2024-09-10	600	House	2	4
Jean Dupont	2024-09-06 01:12:57	2024-09-10	2024-09-15	750	Apartment	3	4.8
Giovanni Rossi	2024-09-06 01:12:57	2024-09-12	2024-09-18	1500	Villa	4	4.2

This query joins 3 tables: Transaction, Booking, and Guest. It retrieves guest payment and booking details, with transaction information, including both payments and refunds, based on the TransactionType field. The results are ordered by the transaction date to display the most recent transactions first.

GuestName	BookingDate	TransactionAmount	TransactionDate	TransactionType	RefundProcessedDate	TransactionDescription
Alessandro Bianchi	2024-09-11 15:08:31	480	2024-10-01 00:00:00	Payment	NULL	Booking for vacation rental
Isabelle Laurent	2024-09-11 15:08:31	525	2024-10-01 00:00:00	Refund	2024-10-01 11:30:00	Full refund for booking canc
Pierre Martin	2024-09-11 15:08:31	800	2024-09-28 00:00:00	Payment	NULL	Booking for vacation rental
Luca Bruni	2024-09-11 15:08:31	800	2024-09-28 00:00:00	Refund	2024-09-28 17:00:00	Refund for booking cancella

```sql
SELECT
    g.Name AS GuestName,
    b.BookingDate,
    t.Amount AS TransactionAmount,
    t.TransactionDate,
    t.TransactionType,
    t.RefundProcessedDate,
    t.Description AS TransactionDescription
FROM Transaction t
JOIN Booking b ON t.BookingID = b.BookingID
JOIN Guest g ON t.GuestID = g.GuestID
ORDER BY t.TransactionDate DESC;
```

```sql
SELECT
    g.Name AS GuestName,
    vr.PropertyType,
    r.Rating AS ReviewRating,
    r.Comment AS ReviewComment,
    cs.IssueDescription AS CustomerServiceIssue,
    cs.Resolution AS CustomerServiceResolution,
    cs.ContactMethod AS CustomerServiceContact
FROM Guest g
JOIN Booking b ON g.GuestID = b.GuestID
JOIN Review r ON b.BookingID = r.BookingID
LEFT JOIN CustomerService cs ON b.BookingID = cs.BookingID
JOIN Room rm ON b.RoomID = rm.RoomID
JOIN VacationRental vr ON rm.VacationRentalID = vr.VacationRentalID
JOIN Host h ON vr.HostID = h.HostID
ORDER BY g.Name ASC;
```

This query joins 6 tables: Guest, Booking, Review, CustomerService, Room, VacationRental, and Host, retrieving guest name, property type, review rating and comment, customer service details, and host information using both inner and left joins on related foreign keys.

GuestName	PropertyType	ReviewRating	ReviewComment	CustomerServiceIssue	CustomerServiceResolution	Custom
Ahmed El-Sayed	Loft	5	Excellent experience!	Noisy neighbors	Spoke to neighbors	Email
Alessandro Bianchi	Apartment	4	Great place, will visit again.	No towels in room	Delivered towels	Phone
Anna Svensson	Townhouse	2	Non è stato come mi aspettavo.	Room too cold	Adjusted thermostat	Phone
Chloe Dubois	Mansion	4	Bellissimo appartamento!	Air conditioning not working	Fixed AC	Phone
Emma Karlsson	House	4	素晴らしい滞在でした！	WiFi not working	Fixed WiFi issue	Email

Data Population & Testing

EventID	BookingID	EventName	StartDate	EndDate	Description
1	1	Berlin Marathon	2024-09-08	2024-09-10	Annual marathon event
2	2	Stockholm Music Festival	2024-09-11	2024-09-13	Music festival in Stockholm
3	3	Paris Fashion Week	2024-09-14	2024-09-20	Fashion event in Paris
4	4	Rome Film Festival	2024-09-21	2024-09-25	Film festival in Rome
5	5	Seoul Food Festival	2024-09-26	2024-09-28	Food festival in Seoul
6	6	Tokyo Game Show	2024-09-29	2024-10-01	Gaming event in Tokyo
7	7	Gothenburg Book Fair	2024-10-02	2024-10-05	Book fair in Gothenburg
8	8	Munich Oktoberfest	2024-10-06	2024-10-10	Beer festival in Munich
9	9	Lyon Lights Festival	2024-10-11	2024-10-13	Light festival in Lyon
10	10	Milan Fashion Week	2024-10-14	2024-10-18	Fashion event in Milan
11	11	Cairo Film Festival	2024-10-19	2024-10-22	Film festival in Cairo
12	12	London Literature Festival	2024-10-23	2024-10-27	Literature festival in London
13	13	Sydney Opera Festival	2024-10-28	2024-11-01	Opera festival in Sydney

Data Population:

Each table was populated with at least 20 entries to ensure that the database could be effectively tested and validated.

Validation and Testing:

Test cases were conducted to verify the integrity of the design, ensuring that the database accurately reflected the ER model and performed as expected.

ReviewID	BookingID	ReviewerID	ReviewerType	Rating	Comment
1	1	1	Guest	5	Fantastisches Erlebnis, werde
2	2	2	Host	4	Bra gäst, men lite bullrigt iblan
3	3	3	Guest	5	Expérience merveilleuse, très
4	4	4	Host	3	Buon soggiorno, ma il Wi-Fi era
5	5	5	Guest	4	훌륭한 숙박이었어요!
6	6	6	Host	5	最高の滞在！また利用したいで
7	7	7	Guest	2	La habitación era más pequeña
8	8	8	Host	4	Локация отличная, хорошие
9	9	9	Guest	5	Wonderful stay, highly recomm
10	10	10	Host	3	Valor razonable, pero con algu

Integrity Check

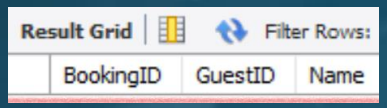
```sql
-- Check for orphaned bookings without a corresponding guest
SELECT b.BookingID, b.GuestID, g.Name
FROM Booking b
LEFT JOIN Guest g ON b.GuestID = g.GuestID
WHERE g.GuestID IS NULL;
```

Procedure:

A SQL query was performed to identify 'Booking' records without valid 'Guest' references using a 'Left Join'.

Ensuring that all bookings are correctly linked to guests prevents orphaned records, which could lead to inconsistent data and errors in reports or queries.
This check helps maintain data reliability across the platform.

Outcome:



The empty result grid confirms all bookings are correctly linked to guests, ensuring data integrity.